ABSTRACT

In this project, each compartment is equipped with a flame sensor, GSM (Global System for Mobile Communication) module, Arduino Uno, and a buzzer. The flame sensor continuously monitors for fire, and if any flames are detected, the system immediately triggers an alarm. The buzzer sounds an alert and at the same time, a notification is sent via SMS to both the Loco Pilot and the Train Traveler Regulators (TTR), specifying the exact compartment where the fire is detected. This real-time alert system allows the authorities to take immediate action, helping to prevent major accidents, injuries, and loss of life.

With this system, even though fires cannot be completely eliminated, the damage can be significantly reduced and quick action can be taken to save lives. My goal with this project is to enhance railway safety and minimize life-threatening situations by ensuring that fire incidents are detected at the earliest possible stage. Through this innovation, we can achieve 99.9% protection against fire-related casualties, making train journeys safer and more reliable for millions of passengers.

INTRODUCTION

Train travel has been an essential mode of transportation in India for over two centuries, with Indian Railways officially operating for more than 160 years. The first passenger train in India ran on April 16, 1853, covering a distance of 34 km between Bori Bunder (Bombay) and Thane. This historic journey was powered by three locomotives—Sahib, Sultan, and Sindh—and comprised thirteen carriages. Over the years, Indian Railways has undergone significant advancements, including electrification, high-speed rail projects, and automation. The first electric train (1500V DC) was introduced on February 3, 1925, operating between Bombay Victoria Terminus and Kurla on the Great Indian Peninsula Railway's (GIPR) Harbour Line. Despite these advancements, train travel remains the most affordable and convenient mode of transportation for millions of Indians. On average, 23 million people travel by train every day, making railway safety a critical concern.

While Indian Railways is equipped with various safety and fire alert systems, fire accidents still occur due to factors such as human negligence, technical failures, derailments, and electrical malfunctions. These incidents jeopardize passenger safety and often lead to significant loss of life and property. Recent incidents highlight the urgent need for improved fire detection and response mechanisms. A fire broke out in three coaches of the Jhansi-bound Taj Express in Delhi, forcing passengers to evacuate immediately. Another fire erupted in an AC coach of a special Holi train near Bihar's Ara, leaving passengers in a state of panic. Although all passengers were safe in these incidents, not every fire-related accident has a fortunate outcome. The current fire alert systems in trains are often insufficient, leading to delayed responses and preventable casualties.

To enhance railway fire safety, my project introduces a compartment-wise fire detection and alert system designed to quickly detect fires, identify the affected compartment, and notify the necessary authorities. Each train compartment will be equipped with flame sensors to detect fire in real time, an Arduino Uno to process sensor data and control the alert system, a GSM module to send immediate notifications to railway authorities, a buzzer to alert passengers inside the affected compartment, and a dedicated display and speaker for the Loco Pilot to provide real-time alerts without requiring mobile phone usage.

The system works by detecting fire through flame sensors, which send signals to the Arduino Uno. The Arduino processes the data and determines the affected compartment. The GSM module then sends an SMS alert to the Loco Pilot and Train Traveler Regulators (TTR), specifying the exact compartment number. A buzzer inside the compartment is activated to warn passengers, while a separate display and alert speaker notify the Loco Pilot, ensuring immediate action without the need for mobile phone usage, as pilots are restricted from using mobile devices while on duty.

Implementing this system will drastically reduce fire-related casualties by enabling faster detection and response. While it may not completely eliminate fire incidents, it will significantly minimize life-threatening situations, reducing casualties by 99.9%. The primary goal of this project is to enhance railway safety, protect passengers, and prevent catastrophic fire accidents in train compartments. With this innovation, we can ensure safer, smarter, and more efficient railway travel for millions of passengers across India.

SYSTEM REQUIREMENTS

The hardware and software specification specifies the minimum hardware and software required to run the system. The hardware configuration specifies below is not by any means the optimal hardware requirements and the performance of the system may be slow on such system.

HARDWARE REQUIREMENT

System type	:	64-bit operating system, x64-based processor
Processor	:	Intel(R) Core(TM) i3-5005U CPU @ 2.00GHz 2.00 GHz
RAM	:	8.00 GB
Hard disk	:	500 GB

SOFTWARE REQUIREMENT

Software is the requirement for any system that undergo software engineering process. Appropriate software may be installed so that the project can be executed with full fledge satisfaction and necessary for the successfully execution of this system is

Operating System	:	Windows 10 Pro
Language	:	Embedded C
Software	:	Arduino IDE

SYSTEM ENVIRONMENT

4.1 SOFTWARE DESCRIPTION

OPERATING SYSTEM

Windows 10 is a Microsoft operating system for personal computers, tablets, embedded devices and internet of things devices. Microsoft released Windows 10 in July 2015 as a follow-up to Windows 8. Windows 10 has an official end of support date of October, 2025, with Windows 11 as its successor

Anyone adopting Windows 10 can upgrade legacy machines directly from Windows 7 or Windows 8 to Windows 10 without re-imaging or performing intrusive and time-consuming system wipes and upgrade procedures. To upgrade from a previous version of Windows 10, IT or users run the Windows 10 OS installer, which transfers any applications and software on the previous OS, as well as settings and preferences over to Windows 10. Organizations and users can pick and choose how they will patch and update Windows 10. IT or users can access a Windows 10 upgrade through the Windows Update Assistant to manually begin an upgrade or wait for Windows Update to offer an upgrade when it is set to run. The Windows user interface was revised to handle transitions between a mouse-oriented interface and a touch screen-optimized interface based on available input devices-particularly on 2-in-1 PCs, both interfaces include an updated Start menu which incorporates element of Windows 7's traditional Start menu with the tiles of Windows 8.

Windows 10 features built-in capabilities that allow corporate IT departments to use mobile device management (MDM) software to secure and control devices running the operating system. In addition, organizations can use traditional desktop management software such as Microsoft System Center Configuration Manager.

One of Windows 10's most notable features is its support for universal apps, an expansion of the Metro-style apps first introduced in Windows 8, Windows 10 also introduced the Microsoft Edge web browser, a virtual desktop system, a window and desktop management feature for enterprise environments, and DirectX 12.

Windows 10 received mostly positive reviews upon its original release in July 2015. Critics praised Microsoft's decision to provide a desktop-oriented interface in line with previous version of Windows, contrasting table-oriented approach of 8, although Windows 10's touch-oriented user interface mode was criticized for containing regressions upon the touch-oriented interface of Windows 8

However, media outlets have been critical of changes to operating system behavior, including mandatory update installation, privacy concerns over data collection performed by the OS for Microsoft and its partners and the adware-like tactics used to promote the operating system on its release.

Windows 10 is used by developers and software engineers for software development, coding, testing, and debugging. It supports development environments like Visual Studio and programming languages like C++, C#, and .NET. Windows 10 is popular among creative professionals in industries such as graphic design, video editing, music production, and photography. It supports creative software like Adobe Creative Cloud, Autodesk, and CorelDraw. Windows 10 IoT editions are utilized in embedded systems, smart devices, kiosks, digital signage, industrial automation, and IoT solutions. It offers security, manageability, and compatibility with loT hardware

ARDUINO

Arduino is an Italian open-source hardware and software company, project, and user community that designs and manufactures single-board microcontrollers and microcontroller kits for building digital devices. Its hardware products are licensed under a CC BY-SA license, while the software is licensed under the GNU Lesser General Public License (LGPL) or the GNU General Public License (GPL), permitting the manufacture of Arduino boards and software distribution by anyone. Arduino boards are available commercially from the official website or through authorized distributors.

Arduino board designs use a variety of microprocessors and controllers. The boards are equipped with sets of digital and analog input/output (I/O) pins that may be interfaced to various expansion boards 'shields' or breadboards for prototyping and other circuits. The boards feature serial communications interfaces, including Universal Serial Bus (USB) on some models, which

are also used for loading programs. The microcontrollers can be programmed using the C and C++ programming languages (Embedded C), using a standard API which is also known as the Arduino Programming Language, inspired by the Processing language and used with a modified version of the Processing IDE. In addition to using traditional compiler tool chains, the Arduino project provides an integrated development environment (IDE) and a command line tool developed in Go.

The Arduino project began in 2005 as a tool for students at the Interaction Design Institute Ivrea, Italy, aiming to provide a low-cost and easy way for novices and professionals to create devices that interact with their environment using sensors and actuators. Common examples of such devices intended for beginner hobbyists include simple robots, thermostats, and motion detectors.

The name Arduino comes from a cafe in Ivrea, Italy, where some of the project's founders used to meet. The bar was named after Arduino of Ivrea, who was the margrave of the March of Ivrea and King of Italy from 1002 to 1014

The Arduino project was started at the Interaction Design Institute Ivrea (IDII) in Ivrea, Italy. At that time, the students used a BASIC Stamp microcontroller at a cost of $50. In 2004, Hernando Barragan created the development platform wiring as a Master's thesis project at IDII, under the supervision of Massimo Banzi and Casey Reas. Casey Reas is known for co-creating, with Ben Fry, the Processing development platform. The project goal was to create simple, low cost tools for creating digital projects by non-engineers. The Wiring platform consisted of a printed circuit board (PCB) with an ATmega128 microcontroller, an IDE based on Processing and library functions to easily program the microcontroller.[5] In 2005, Massimo Banzi, with David Mellis, another IDII student, and David Cuartielles, extended Wiring by adding support for the cheaper ATmega8 microcontroller. The new project, forked from Wiring, was called Arduino.

The initial Arduino core team consisted of Massimo Banzi, David Cuartielles, Tom Igoe, Gianluca Martino, and David Mellis. Following the completion of the platform, lighter and less expensive versions were distributed in the open-source community. It was estimated in mid-2011

that over 300,000 official Arduino's had been commercially produced, and in 2013 that 700,000 official boards were in users' hands.

ARDUINO IDE

Arduino ide software The Arduino Integrated Development Environment - or Arduino Software (IDE) - contains a text editor for writing code, a message area, a text console, a toolbar with buttons for common functions and a series of menus. It connects to the Arduino hardware to upload programs and communicate with them.



Figure 4.1.1 Arduino IDE

INSTALL ARDUINIO IDE SOFTWARE

Step 1: Visit the Official Arduino Website

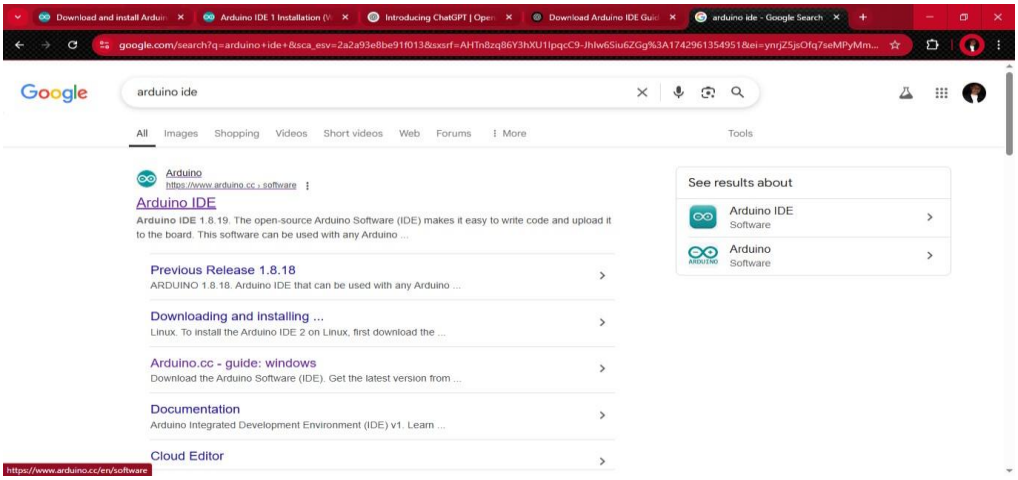


Figure 4.1.2 Official Page of Arduino IDE

Step2: Click on the download Link

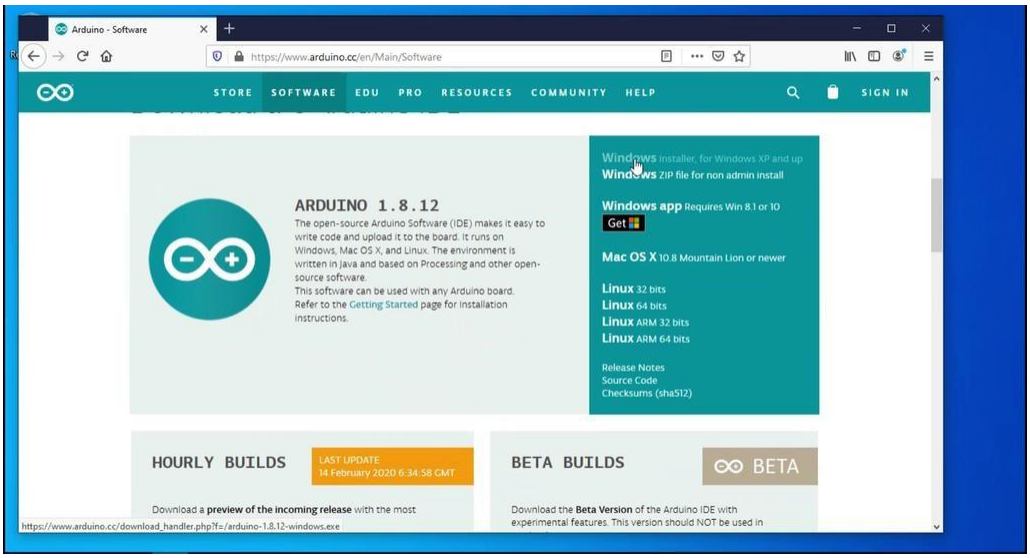


Figure 4.1.3 Download link

Scroll down until you see the link that says 'Windows installer' and click on it.

Step 3: Begin The download

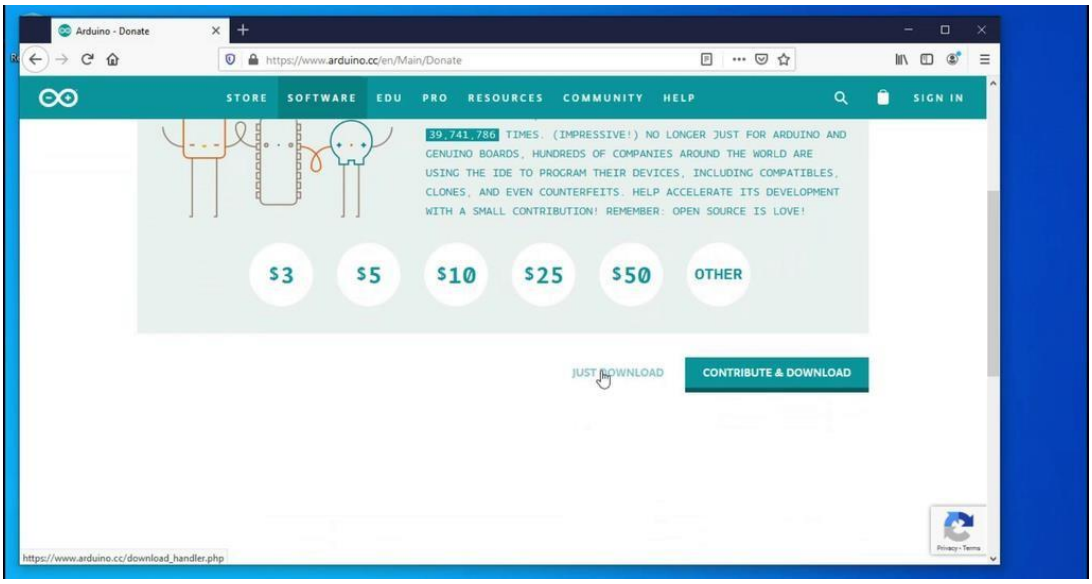


Figure 4.1.4 Download Starts

After clicking on the download link you'll be redirected to the donation page, here you can donate or skip it if you like by clicking on the 'Just download' link.

Step 4: Begin the Installation Process

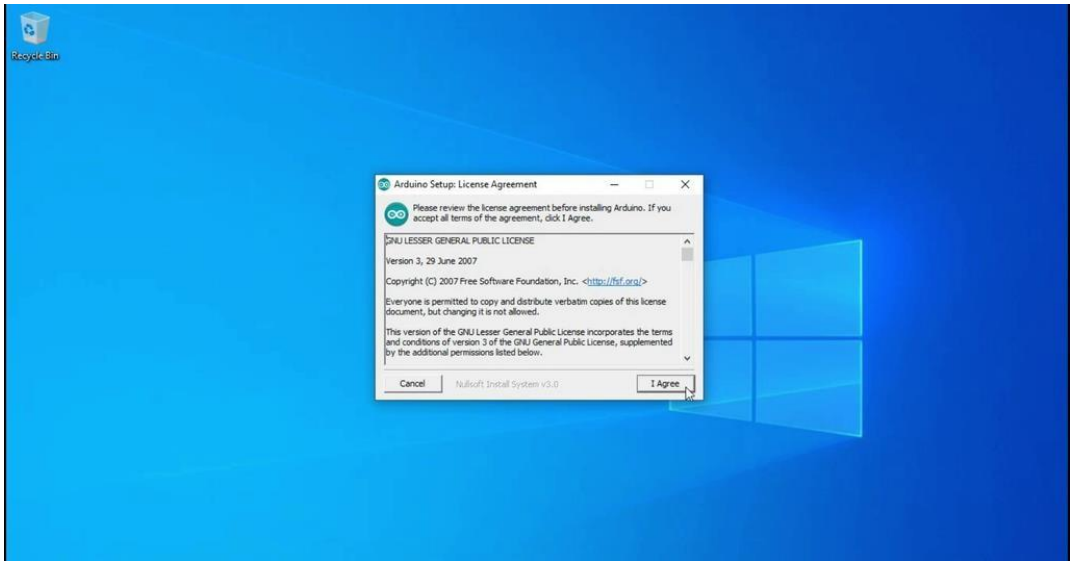


Figure 4.1.5 Installation Process

Open the downloaded file, A new window will open asking you to agree to the license agreement. Click on 'I agree' to continue

Step5: Select what to install

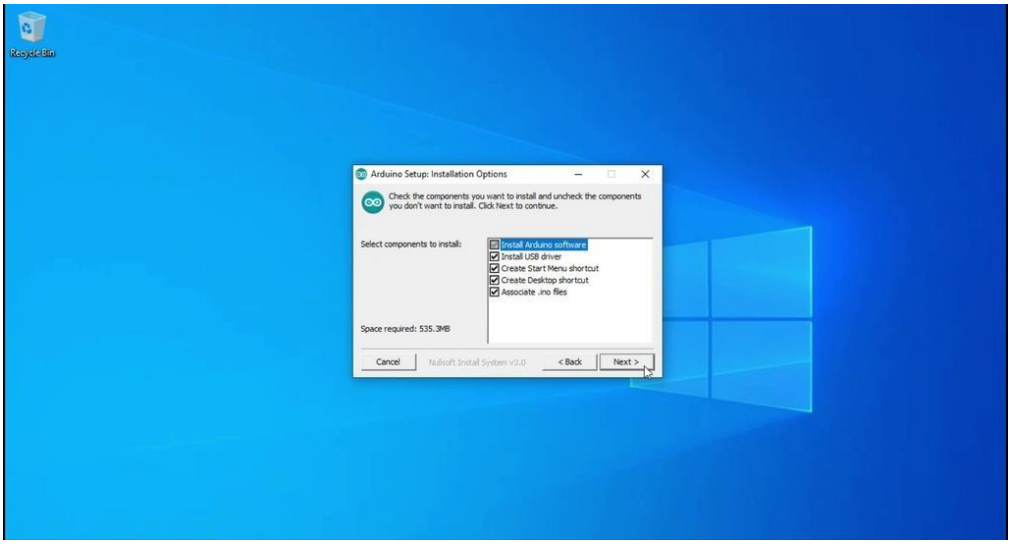


Figure 4.1.6 Installation Process

Now you'll see all the available options to install the software with, If you don't know what you need, it is best to keep everything checked as you can change it later when the installation has finished.

Click on 'Next' to continue.

Step 6: Choose the Installation path

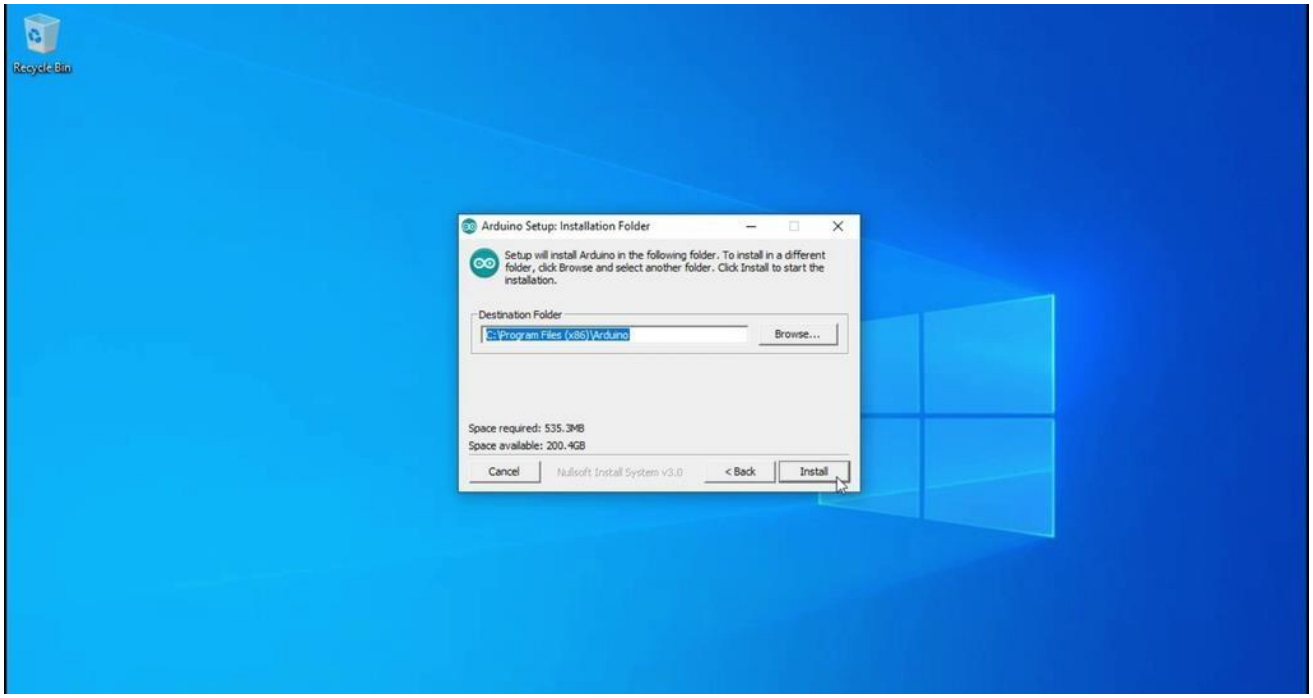


Figure 4.1.7 Install Path

Now you have to choose the path the software will be installed in, it is fine to leave it at the configured location but if you want the Arduino IDE somewhere else installed you can change that here, Click on 'Install' to begin the installation

Step 7: Finish the Installation

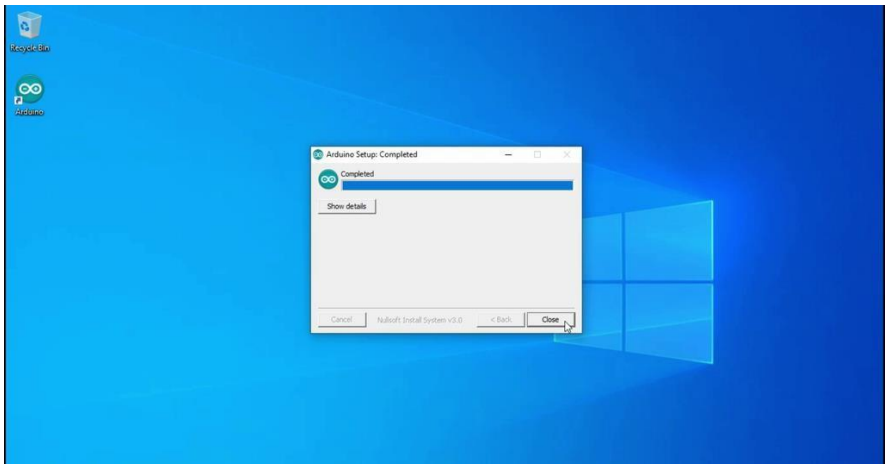


Figure 4.1.8 Finish Installation

Wait until the installation is finished, it shouldn't take very long, When the installation is finished you may click on 'close' to end the setup wizard.

Step 8: Launch the Arduino IDE

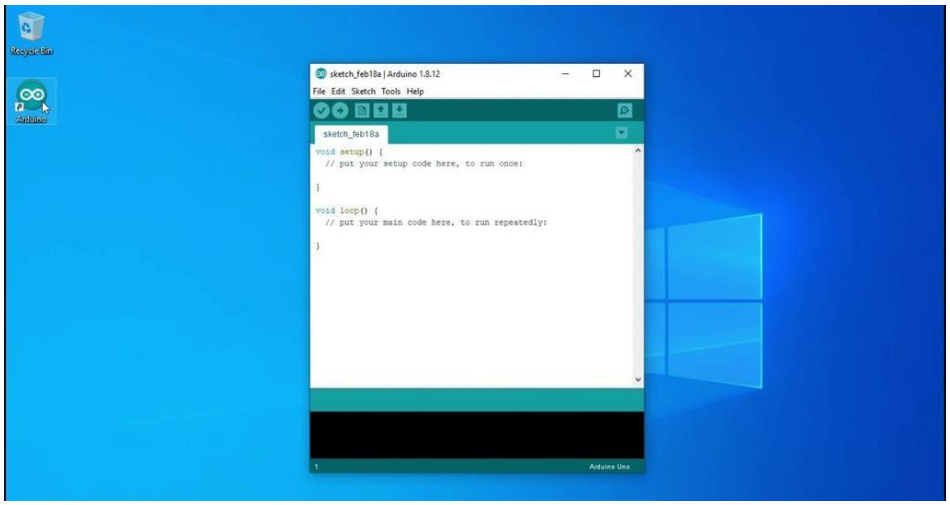


Figure 4.1.9 Launch the Arduino IDE

The Arduino IDE has now been successfully installed, To launch the IDE you can click on the Desktop icon that was created for you, or by searching for it in the start menu.

EMBEDDED C

Embedded C is a programming language that is used in the development of Embedded Systems. Embedded Systems are specialized systems designed to perform very specific functions or tasks. Embedded System is the combination of hardware and software, and the software is generally known as firmware which is embedded into the system hardware. Embedded C is used to program a wide range of microcontrollers and microprocessors. Embedded C requires less number of resources to execute in comparison with high-level languages such as assembly programming language.

Embedded C has some additional data types and keywords. There are some special data types in Embedded C like sbit, sfr which are used for addressing special function registers in memory. Embedded C allows us to work with hardware devices like sensors, and input-output devices. There are various Embedded C compilers to compile the embedded C program such as Keil Compiler, SPJ Compiler, Embedded GNU C Compiler, etc. Embedded Systems can be classified into small-scale, medium-scale, and sophisticated embedded systems. The devices like air conditioners, printers, and mobile phones that we use in our daily lives are programmed by embedded C.

KEY CHARACTERISTICS OF EMBEDDED C

Efficiency: In Embedded C we can create an efficient code to optimize the limited resources available in embedded systems. It aims to minimize memory usage and maximize performance.

Direct Hardware Interaction: Embedded C allows programmers to interact directly with hardware components, such as microcontrollers, sensors, actuators, and other peripherals. This direct interaction facilitates precise control over the hardware, critical in embedded applications.

Low-level Programming: Embedded C involves low-level programming, which deals with hardware-specific details like memory addresses, I/O ports, and register manipulation. This level of control is essential for efficiently managing hardware resources.

Real-time Operations: Embedded systems often operate in real-time environments, requiring precise timing and response to events. Embedded C allows programmers to handle real-time tasks efficiently.

STRUCTURE OF EMBEDDED C PROGRAM

Comments: Comments are readable text written to help user understand the code easily. They are ignored by compiler and do not take up any memory in the final code. There are two types of comments, Single line comments and Multiline comments.

Preprocessor Directive: In Embedded C Preprocessor Directives are represented using ***#include*** or **#define**. Preprocessor Directives are used to indicate a header file specific to a microprocessor or microcontroller which contains all the functions, SFR's and the bits in those SFR's. reg51 header file is used in case of 8051 microcontroller.

Global Variables: Global variables as the name suggests are global to program that is they can be accessed anywhere in the program. Global variables are static variables and are placed in RAM memory locations.

Local Variables: Local variables in contrast to global variables are confined to their respective functions. Normally these variables are placed in stack or registers. It is only valid within the function in which it is declared.

Function: Function is a group of statements that together performs a task. A function declaration tells the compiler about the name, return type and parameter of the function. A function definition provides actual body of the function.

Main Function: Every Embedded C program has one main function and may contain one or more functions in the main functions. The program execution starts from the main function, and it is a core of every execution. If more than one main function is written in the code, then compiler will confuse from where to start the program execution.

ADVANCED TECHNIQUES FOR EMBEDDED C

Pointer manipulation: Pointers in C are powerful but can be complex. They allow direct access to memory locations, aiding in efficient data manipulation. Understanding pointer arithmetic, dynamic memory allocation (malloc/free), and using pointers accessing hardware registers or structures are crucial in embedded C programming.

Interrupt handling: In embedded systems, interrupts are used to handle asynchronous events. Mastering ISR involves understanding how to write interrupt service routines, handle interrupt priorities, manage shared resources, and minimize interrupt latency to ensure timely response to events.

RTOS (Real-Time Operating Systems): RTOS facilitates multitasking within embedded systems. Understanding concepts like task scheduling, context switching, inter-process communication (IPC), and synchronization mechanisms (semaphores, mutexes) is very much important for developing real-time embedded applications.

Peripheral Interfacing: Embedded systems interact with various peripherals. Knowledge of communication protocols (UART, SPI, I2C), handling GPIO pins, configuring timers, and managing interrupts related to peripherals is important for effective interfacing.

Low-power Optimization: Embedded devices often run on limited power. Techniques to reduce power consumption involve utilizing low-power modes provided by microcontrollers, selectively shutting down unused peripherals, and optimizing algorithms for energy efficiency.

Memory Management: Embedded systems have limited memory. Efficiently managing memory includes minimizing memory fragmentation, choosing appropriate data types, implementing memory pooling, and handling dynamic memory allocation carefully to avoid memory leaks.

Debugging and Testing: Embedded system debugging involves using hardware debuggers, emulators, simulators, and printf-style debugging.

ADVANTAGES OF WRITING A PROGRAM IN EMBEDDED C

- It is easy and less time consuming to write code in Embedded C instead of assembly programming language.
- Embedded C program is easier to modify and update..

DISADVANTAGES OF EMBEDDED C PROGRAMMING

- It is not scalable scalability is a problem with Embedded C.
- It has limitations such as restricted RAM which affects the computer's compatibility.

WRITING SKETCHES

Programs written using Arduino Software (IDE) are called sketches. These sketches are written in the text editor and are saved with the file extension .ino. The editor has features for cutting/pasting and for searching/replacing text. The message area gives feedback while saving and exporting and also displays errors. The console displays text output by the Arduino Software (IDE), including complete error messages and other information. The bottom right hand corner of the window displays the configured board and serial port. The toolbar buttons allow you to verify and upload programs, create, open, and save sketches, and open the serial monitor.

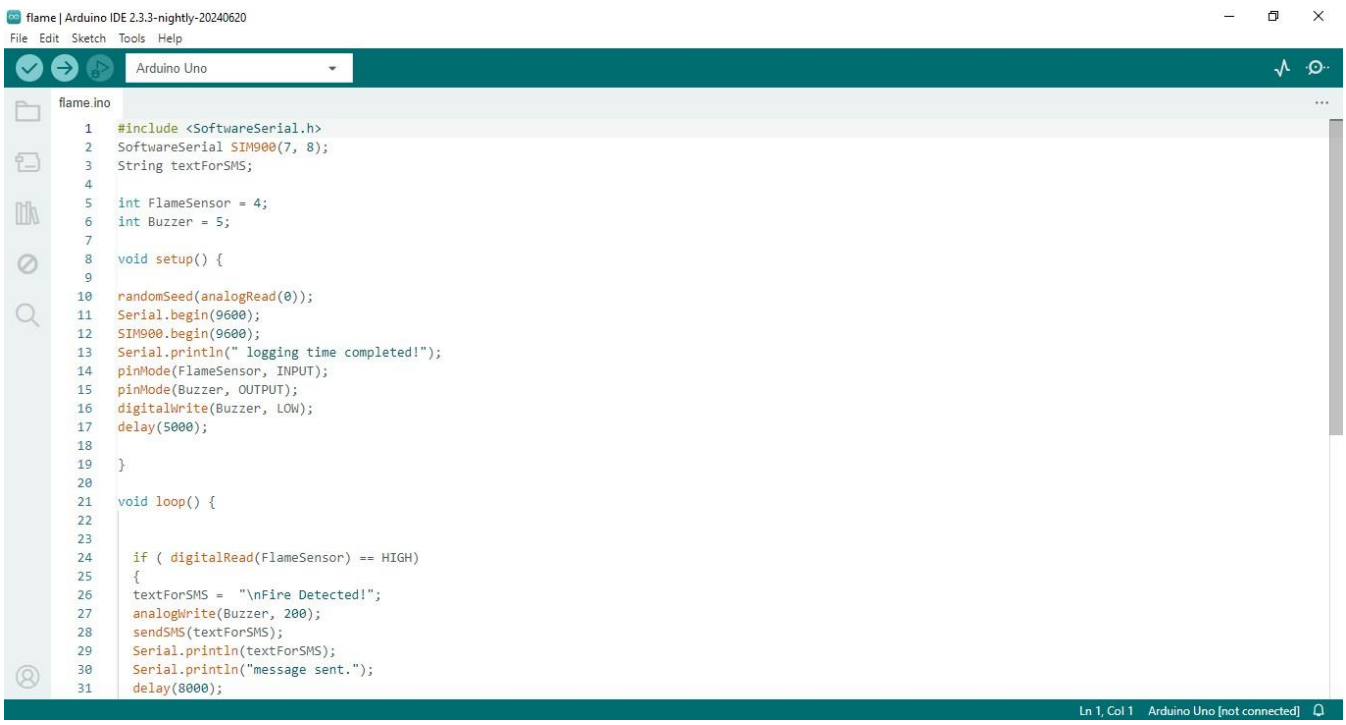


Figure 4.1.10: Sketches

Verify Checks your code for errors compiling it.



Upload Compiles your code and uploads it to the configured board. See uploading below for details.



Create a new sketch.



Open presents a menu of all the sketches in your sketchbook. Clicking one will open it within the current window overwriting its content.



To save your sketch



Serial Monitor Opens the serial monitor.



4.2 HARDWARE DESCRIPTION

ARDUINO UNO

The Arduino Uno is a microcontroller board that was developed in 2005 at the Interaction Design Institute Ivrea (IDII) in Italy. The Arduino Uno was created to be a simple, inexpensive, and easy-to-program device for non-specialists to interact with other electric components

The Arduino Uno is a series of open-source microcontroller board based on a diverse range of microcontrollers (MCU). It's initially developed and released by Arduino Company in 2010.The microcontroller board is equipped with sets of digital and analog input/output (I/O)

pins that may be interfaced to various expansion boards (shields) and other circuits. The board has 14 digital I/O pins (six capable of PWM output), 6 analog I/O pins, and is programmable with the Arduino IDE (Integrated Development Environment), via a type B USB cable. It can be powered by a USB cable or a barrel connector that accepts voltages between 7 and 20 volts, such as a rectangular 9-volt battery. It has the same microcontroller as the Arduino Nano board, and the same headers as the Leonardo board. The hardware reference design is distributed under a Creative Commons Attribution Share-Alike 2.5 license and is available on the Arduino website. Layout and production files for some versions of the hardware are also available.

The word "Uno" means "one" in Italian and was chosen to mark a major redesign of the Arduino hardware and software. The Uno board was the successor of the Duemilanove release and was the 9th version in a series of USB-based Arduino boards Version 1.0 of the Arduino IDE for the Arduino Uno board has now evolved to newer releases. The ATmega328 on the board comes preprogrammed with a boot loader that allows uploading new code to it without the use of an external hardware programmer.

While the Uno communicates using the original STK500 protocol, it differs from all preceding boards in that it does not use a FTDI USB-to-UART serial chip. Instead, it uses the Atmega16U2 (Atmega8U2 up to version R2) programmed as a USB-to-serial converter.



Figure 4.1.11 Arduino UNO

Arduino UNO is a low-cost, flexible, and easy-touse programmable open-source microcontroller board that can be integrated into a variety of electronic projects. The board is

equipped with sets of digital and analog input/output pins that may be interfaced to various expansion boards and other circuits.

DIFFERENT COMPONENTS IN ARDUINO UNO

- USD Jack
- Power Jack
- Voltage Regulator
- Crystal Oscillator
- Reset Button
- Power Pins
- Analog Input Pins
- Digital Pins
- TX and RX Pins
- Power on LED
- LED attached to digital pin 13
- AREF
- ICSP Pins
- Main Microcontroller

MAIN MICROCONTROLLER



Figure 4.1.12 Main Microcontroller

Each arduino board has its own microcontroller – brain of the board and its usually of the ATMEL company ATMEGA 328p used to store data and run program.

USB JACK



Figure 4.1.13 USB Jack

Used to upload programs from computer to Arduino board and also used to power up the Arduino board

POWER JACK

Used to power Arduino Input voltage Range 7V to 12V



Figure 4.1.14 Power Jack

WAYS OF CONNECTION

- Rechargeable Batteries
- Disposable Batteries
- Wall – adapters
- Solar panel

VOLTAGE REGULATOR



Figure 4.1.15 Voltage regulators

Used to control & stabilize the voltage given to the arduino board and used by the processor and other element

CRYSTAL OSCILLATOR

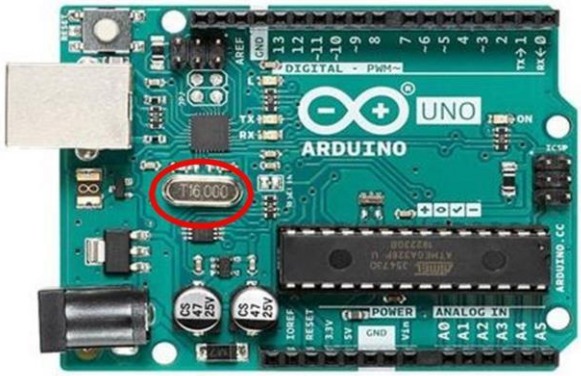


Figure 4.1.16 Crystal Oscillator

Helps Arduino in dealing with timing requirement and its used to calculate time Its frequency - 16MHz

RESET BUTTON



Figure 4.1.17 Reset Button

Used to start our program from the beginning

POWER PINS

- 3.3V – Supply 3.3 output volt
- 5V – Supply 5 output volt



Figure 4.1.18 Power Pins

- GNG – Used to ground our circuits
- Vin – Can also be used to power the Arduino board from an external power source

ANALOG INPUT PINS

Six analog input pins (A0 – A5) Used to read the signal from an analog sensor



Figure 4.1.19 Analog Input

Can also be used ad digital o/p, i/p pins like digital pins.

DIGITAL PINS



Figure 4.1.20: Digital Pins

14 Digital pins – 0 to 13 here 6 pins provide PWM (Pulse width Modulation) output when set I/P: these pins can read voltage and when set as O/P: these pins can apply voltage

TX AND RX PINS



Figure 4.1.21 TX and RX pins

TX – Transmit | RX – Receive Arduino uses these pins to communicate with other electronics via serial communication and avoid using these pins for other tasks other than serial communication unless you're running out of pins

LED ATTACHMENT TO DIGITAL PIN 13



Figure 4.1.22 LED Attachment

Useful for an easy debugging of the Arduino sketch

AREF (ANALOG REFERENCE)



Figure 4.1.23 Analog Reference

Stands for Analog reference It is sometimes, used to set an external reference voltage (Between 0 to 5V) as the upper limit for the analog input pins

ICSP PINS



Figure 4.1.24 ICSP Pins

In circuit serial programming one of the several methods available for programming Arduino boards SPI Communication protocols



GSM (GLOBAL SYSTEM FOR MOBILE COMMUNICATION)

Developed by group speciale mobile(founded 1892) which was an initiative of CPET(Conference of European Post and Telecommunication) Under ETSI,GSM is named as ”Global System for Mobile Communication” in 1989 Full set of specifications phase-I became available in 1990 Phase 2 of the GSM specifications occurs in 1995.Coverage is extended to rural areas

Global System For Mobile(GSM) is a second generation cellular standard developed to cater voice services and data delivery using digital modulation GSM digitizes and compresses data , then sends it don a channel with two other streams of user data ,each in its own time slot. It operates at either the 900 megahertz (MHZ) or 1800 MHZ frequency band



Figure 4.1.25 GSM Module

GSM stands for Global System for Mobile Communication. It is a digital mobile network commonly utilized by mobile phone users around the world. It is the most popular of the three digital wireless telephony systems (TDMA, GSM, and CDMA) and uses the combination of FDMA and TDMA. It uses 4 different frequency bands 850 MHz, 900 MHz, 1800 MHz, and 1900 MHz GSM converts and compresses data before sending it along a channel with two other streams of user data, each with its time slot.

GSM HAVE 4 DIFFERENT SIZES OF CELLS

- Macro: In this size of the cell, a Base Station antenna is installed.
- Micro: In this size of cell, antenna height is less than the average roof level.
- Pico: Small cells' diameter of a few meters.
- Umbrella: It covers the shadowed (Fills the gaps between cells) regions**.**

FEATURES OF GSM

- Supports international roaming
- Clear voice clarity
- Ability to support multiple handheld devices.
- Spectral / frequency efficiency

- Low powered handheld devices.
- Ease of accessing network
- International ISDN compatibility.
- Low service cost.
- New features and services.

THE ARCHITECTURE OF GSM

- BSS: BSS stands for Base Station Subsystem. BSS handles traffic and signaling between a mobile phone and the network switching subsystem. BSS having two components BTS and BSC.
- NSS: NSS stands for Network and Switching Subsystem. NSS is the core network of GSM. That carried out call and mobility management functions for mobile phone present in network. NSS have different components like VLR, HLR and EIR.
- OSS: OSS stands for Operating Subsystem. OSS is a functional entity which the network operator Monitor and control the system. OMC is the part of OSS. Purpose of OSS is to offer the customer cost-effective support for all GSM related maintenance services

Suppose there are 3 Mobile stations which are connected with the tower and that tower is connected to BTS through TRX, and then further connected to BSC and MSC. Let's understand the functionality of different components.

MS: MS stands for Mobile System. MS comprises user equipment and software needed for communication with a mobile network. Mobile Station (MS) = Mobile Equipment (ME) + Subscriber Identity Module (SIM). Now, these mobile stations are connected to tower and that tower connected with BTS through TRX. TRX is a transceiver which comprises transmitter and receiver. Transceiver has two performances of sending and receiving.

BTS: BTS stands for Base Transceiver Station which facilitates wireless communication between user equipment and a network. Every tower has BTS.

BSC: BSC stands for Base Station Controller. BSC has multiple BTS. You can consider the BSC as a local exchange of your area which has multiple towers and multiple towers have BTS.

MSC: MSC stands for Mobile Switching Center. MSC is associated with communication switching functions such as call setup, call release and routing. Call tracing, call forwarding all

functions are performed at the MSC level. MSC is having further components like VLR, HLR, AUC, EIR and PSTN.

VLR: VLR stands for Visitor Location Register. VLR is a database which contains the exact location of all mobile subscribers currently present in the service area of MSC. If you are going from one state to another state then your entry is marked into the database of VLR.

HLR: HLR stands for Home Location Register. HLR is a database containing pertinent data regarding subscribers authorized to use a GSM network. If you purchase SIM card from in the HLR. HLR is like a home which contains all data like your ID proof, which plan you are taking, which caller tune you are using etc.

AUC: AUC stands for Authentication Center. AUC authenticates the mobile subscriber that wants to connect in the network.

EIR: EIR stands for Equipment Identity Register. EIR is a database that keeps the record of all allowed or banned in the network. If you are banned in the network then you can't enter the network, and you can't make the calls.

PSTN: PSTN stands for Public Switched Telephone Network. PSTN connects with MSC. PSTN Originally a network of fixed line analog telephone systems. Now almost entirely digital in its core network and includes mobile and other networks as well as fixed telephones. The earlier landline Phones which places at our home is nothing but PSTN.

OMC: OMC stands for Operation Maintenance Center. OMC monitor and maintain the performance of each MS, BSC and MSC within a GSM system.

Three subsystem BSS, NSS and OSS are connected with each other via some interfaces. Total three interfaces are there:

Air Interface: Air interface is also known as UM interface. Interface between MS and BTS is called as UM interface because it is mobile analog to the U interface of ISDN.

Abis Interface: It is a BSS internal interface linking with BTS and BSC.

An interface: It provides communication between BSS and MSC

GSM WORKS

GSM describe the procedures for second-generation digital mobile networks, such as those used by mobile phones. It is a broad-area communications technology program that uses digital radio channeling to provide audio, information, and multimedia communication systems. Every GSM radio channel is 200 kHz broad and is further divided into frames of eight time slots. The GSM system consists of mobile stations, base stations, and interweaving switching systems.

The GSM program allows 8 to 16 audio users to share a single radio channel, and each radio transmission station can have numerous radio channels. Because of its simplicity, cost, and accessibility, GSM is now the most often utilized network technology in the Internet of Things (IoT).

APPLICATIONS OF GSM

- Mobile Telephony
- VoIP Integration
- SMS (Short Message Service)
- Mobile Banking
- Smart Home Systems
- Surveillance Systems
- Alarm System
- Cell Broadcasting

ADVANTAGES OF GSM

Compatibility: GSM is widely used around the world, so it is compatible with many different networks and devices.

Security: GSM offers enhanced security features such as authentication, encryption and confidentiality which help to protect the user's privacy and data.

Efficient use of bandwidth: GSM uses a time-division multiplexing (TDM) technique which enables many users to share the same frequency channel at different times, making it an efficient use of the available bandwidth.

Roaming: GSM allows users to roam internationally and use their mobile phones in other countries that use the same GSM standard.

Wide range of features: GSM supports a wide range of features, including call forwarding, call waiting, voicemail, conference calling, and more.

DISADVANTAGES OF GSM

Limited coverage: GSM networks may have limited coverage in some remote areas, which can make it difficult for users to make calls or access the internet.

Network congestion: GSM networks may become congested during peak hours, which can lead to dropped calls or poor call quality.

Security vulnerabilities: Although GSM offers enhanced security features, it is still vulnerable to certain types of attacks, such as eavesdropping and spoofing.

Data transfer speed: GSM networks offer relatively slow data transfer speeds compared to newer technologies such as 3G and 4G.

Limited capacity: GSM networks have a limited capacity for handling large volumes of data, which can be a disadvantage for users who require high-speed internet access or other data-intensive applications.

SERVICES OF GSM

Bearer services/ data services: GSM specifies different mechanism for data transmission, The original GSM allowing for data rates of up to 9600 bits/s. Bearer services permit transparent or non transparent data transmission.

Transparent bearer services: Transparent bearer services only use the physical layer to transmit data. Data transmission has a constant delay at throughput if no transmission error occurs.

Non-transparent bearer services: Non-transparent bearer services use protocols of layer two and three two three to implement error correction and flow control (data link layer and network layer).

TELE SERVICES SUCH AS:

- Video calls and face emoji.
- Short text message (SMS).
- Supplementary services are advanced services such as:
- Conference calls.
- Call waiting.
- Call forwarding.

GSM security: GSM offers security using confidential information stored in the AUC and in the individual SIM. The SIM stores personal secret data and is protected with a pin against unauthorized use.

FLAME SENSOR

A Flame Sensor, also known as a flame detector or fire sensor, is a specialized sensor designed to detect the presence of flames or fire. It plays a crucial role in fire safety systems by providing early detection of fires or flames, which allows for prompt response measures to mitigate potential hazards. Here's an introduction to flame sensors



Figure 4.1.26 Flame Sensor

Flame sensors are equipped with IR or UV sensors that can identify the unique light signatures emitted by flames. When a flame is present, it emits light in specific wavelengths within the IR or UV spectrum. The sensor is designed to recognize these characteristic wavelengths associated with flames, distinguishing them from background light sources. Upon detecting the signature of a flame, the sensor generates an electrical signal, which can be used to

activate alarms, trigger safety systems, or initiate other responses. By filtering out non-flame sources of IR or UV light, such as sunlight or artificial lighting, flame sensors minimize false alarms and provide reliable flame detection in various applications.

Function: A phototransistor is a type of light-sensitive semiconductor device that detects light and converts it into an electrical signal. It operates similarly to a regular transistor but uses light instead of electrical signals to control current flow between its terminals.

The LM393: is a widely used dual comparator integrated circuit (IC) that can also be relevant in the context of flame sensors, especially in applications where precise comparison of signals is required. Here's an overview of the LM393 IC and its potential role in a flame sensor system:

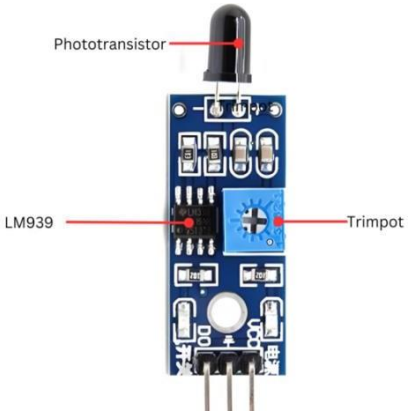


Figure 4.1.27 Flame Sensor Components

Sensitivity Adjustment: A trimpot can be used to adjust the sensitivity of the sensor circuitry. For example, in a UV flame sensor using a phototransistor, the trimpot could adjust the threshold at which the sensor detects UV light emitted by the flame.

TECHNICAL SPECIFICATIONS

- Operating voltage 3.3V-5V.
- Comparator chip LM393 makes module readings stable.
- Fast Response Time
- Sensitivity adjustable
- Detects a flame or a light source of a wavelength in the range of 760nm-1100 nm.
- Detection range: up to 100 cm.
- Adjustable detection range.
- Detection angle about 60 degrees, it is sensitive to the flame spectrum.

PINOUT

- VCC: Connects to a positive voltage supply (e.g., 5V or 3.3V).
- GND: Connects to the ground (0V) of the power supply.
- OUT: Outputs a digital signal indicating the presence of a flame (HIGH when flame detected, LOW otherwise).

LIQUID CRYSTAL DISPLAY (LCD)

LCD (Liquid Crystal Display) is a type of flat panel display which uses liquid crystals in its primary form of operation. LEDs have a large and varying set of use cases for consumers and businesses, as they can be commonly found in smart phones, televisions, computer monitors and instrument panels.

The LCDs have a parallel interface, meaning that the microcontroller has to manipulate several interface pins at once to control the display. The interface consists of the following pins:

- A register select (RS) pin that controls where in the LCD's memory you're writing data to. You can select either the data register, which holds what goes on the screen, or an instruction register, which is where the LCD's controller looks for instructions on what to do next.
- A Read/Write (R/W) pin that selects reading mode or writing mode
- An Enable pin that enables writing to the registers

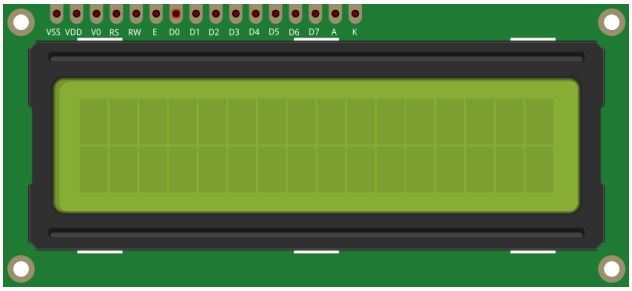


Figure 4.1.28 LCD Display

8 data pins (D0 -D7). The states of these pins (high or low) are the bits that you're writing to a register when you write, or the values you're reading when you read.

There's also a display contrast pin (Vo), power supply pins (+5V and GND) and LED Backlight (Bklt+ and BKlt-) pins that you can use to power the LCD, control the display contrast, and turn on and off the LED backlight, respectively.

The process of controlling the display involves putting the data that form the image of what you want to display into the data registers, then putting instructions in the instruction register. The Liquid Crystal Library simplifies this for you so you don't need to know the low-level instructions.

The Hitachi-compatible LCDs can be controlled in two modes: 4-bit or 8-bit. The 4-bit mode requires seven I/O pins from the Arduino, while the 8-bit mode requires 11 pins. For displaying text on the screen, you can do most everything in 4-bit mode, so example shows how to control a 16x2 LCD in 4-bit mode.

BUZZER

There are many ways to communicate between the user and a product. One of the best ways is audio communication using a buzzer IC. So during the design process, understanding some technologies with configurations is very helpful. So, this article discusses an overview of an audio signaling device like a beeper or a buzzer and its working with applications

An audio signaling device like a beeper or buzzer may be electromechanical or piezoelectric or mechanical type. The main function of this is to convert the signal from audio to sound. Generally, it is powered through DC voltage and used in timers, alarm devices, printers, alarms, computers, etc. Based on the various designs, it can generate different sounds like alarm, music, bell & siren.



Figure 4.1.29 Buzzer

The pin configuration of the buzzer is shown below. It includes two pins namely positive and negative. The positive terminal of this is represented with the '+' symbol or a longer terminal. This terminal is powered through 6Volts whereas the negative terminal is represented with the '-'symbol or short terminal and it is connected to the GND terminal.

TYPES OF BUZZER

A buzzer is available in different types which include the following.

- Piezoelectric
- Electromagnetic
- Mechanical
- Electromechanical
- Magnetic

PIEZOELECTRIC

As the name suggests, the piezoelectric type uses the piezoelectric ceramic's piezoelectric effect & pulse current to make the metal plate vibrate & generate sound. This kind of buzzer is made with a resonance box, multi resonator, piezoelectric plate, housing, impedance matcher, etc. Some of the buzzers are also designed with LEDs. The multi resonator of this mainly includes ICs and transistors. Once the supply is given to this resonator, it will oscillate and generates an audio signal with 1.5 to 2.kHz. The impedance matcher will force the piezoelectric plate to produce sound.

ELECTROMAGNETIC

This type of buzzer is made with a magnet, solenoid coil, oscillator, housing, vibration diaphragm, and magnet. Once the power supply is given, the oscillator which produces the audio signal current will supply throughout the solenoid coil to generate a magnetic field.Sometimes, the vibration diaphragm will vibrate & generates sound under the magnet & solenoid coil interaction. The frequency range of this ranges from 2 kHz to 4kHz.

MECHANICAL

These types of buzzers are subtypes of electromagnetic, so the components used in this type are also similar. But the main difference is that the vibrating buzzer is placed on the outside instead of the inside.

ELECTROMECHANICAL

The designing of these types of buzzers can be done with a bare metal disc & an electromagnet. The working principle of this is similar to magnetic and electromagnetic. It generates sound throughout the disc movement & magnetism.

MAGNETIC

Like a piezo type, magnetic is also used to generate a sound but they are different due to core functionality. The magnetic type is more fixed as compared to the piezo type because they work through a magnetic field. Magnetic buzzers utilize an electric charge instead of depending on piezo materials to generate a magnetic field, after that it permits another element of the buzzer

to vibrate & generate sound. The applications of magnetic buzzers are similar to the piezo type in household devices, alarms such as watches, clocks & keyboards.

JUMPER WIRES

Jumper wires are electrical wires used to establish temporary or permanent connections between components in electronic circuits. They are widely used in prototyping, breadboarding, and circuit testing. Typically insulated with plastic or rubber, jumper wires contain a conductive metal core, usually copper, which allows electrical signals to pass through with minimal resistance.

Jumper wires come with different connectors at their ends, which determine their compatibility with various types of components, such as breadboards, microcontrollers, and sensors. They eliminate the need for soldering, making them ideal for quick modifications and debugging in circuit designs.

TYPES OF JUMPER WIRES

There are different types of jumper wires. Some have the same type of electrical connector at both ends, while others have different connectors. Some common connectors are:

Solid tips: are used to connect on/with a breadboard or female header connector. The arrangement of the elements and ease of insertion on a breadboard allows increasing the mounting density of both components and jump wires without fear of short-circuits. The jump wires vary in size and colour to distinguish the different working signals.

Crocodile clips: are used, among other applications, to temporarily bridge sensors, buttons and other elements of prototypes with components or equipment that have arbitrary connectors, wires, screw terminals, etc.

Banana connectors: are commonly used on test equipment for DC and low-frequency AC signals.

Registered jack (RJnn): are commonly used in telephone (RJ11) and computer networking (RJ45).

RCA connectors: are often used for audio, low-resolution composite video signals, or other low-frequency applications requiring a shielded cable.

RF connectors: are used to carry radio frequency signals between circuits, test equipment, and antennas.

RF jumper cables: Jumper cable is a smaller and more bendable corrugated cable which is used to connect antennas and other components to network cabling. Jumpers are also used in base stations to connect antennas to radio units. Usually the most bendable jumper cable diameter is 1/2".

There are three types of jumper wire: Female-to-Female, Male-to-Male, and Male-to-Female. The reason we call it Male-to-Female is because it has the outstanding tip in one end as well as a sunk female end. Male-to-Male means both side are male and Female-to-Female means both ends are female.

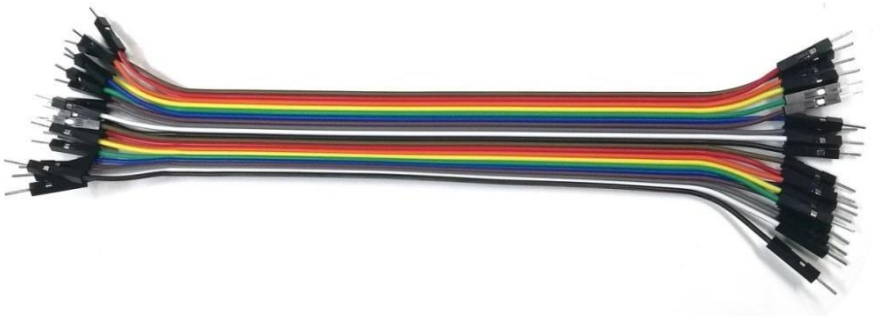


Figure 4.1.31 Jumper Wire

More than one type of them may be used in a project. The color of the jump wires is different but it doesn't mean their function is different accordingly; it's just designed so to better identify the connection between each circuit.

MALE-TO-MALE JUMPER WIRES

Male-to-male jumper wires are the most common type of jumper wires. Each end of the wire has an exposed metal pin that fits into female sockets, such as those found on breadboards or microcontrollers. These wires facilitate quick circuit assembly and allow multiple components to be interconnected without requiring soldering.

APPLICATIONS OF MALE-TO-MALE JUMPER WIRES INCLUDE:

- Connecting sensors and actuators to development boards.
- Building temporary circuits for testing.
- Interconnecting modules with breadboards.

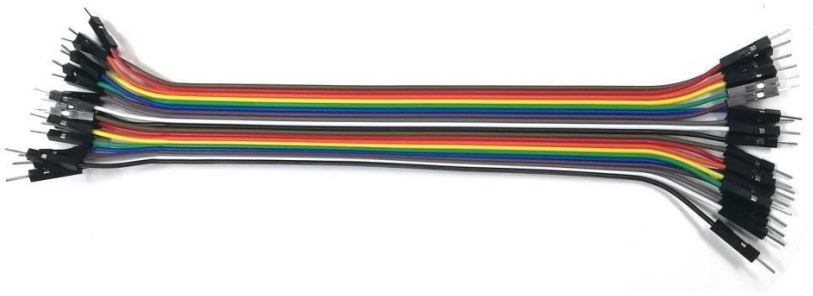


Figure 4.1.32 Male to Male Jumper wire

Jumper wires are handy tools for electronic prototyping. The connector pins on one end connect two points without soldering. They are generally used with breadboards and prototyping tools such as Arduino. Because they come in various colours and tiny details, they are a versatile and necessary component for electronic projects. Make sure you have enough jumper wires before starting your next project.

MALE TO FEMALE JUMPER WIRES

Male-to-female jumper wires are insulated conductive wires with a male connector (pin) on one end and a female connector (socket) on the other. This makes them ideal for connecting components, modules, or breadboards in an electrical circuit. The male end is usually intended to plug into a female header or socket, while the female end is to accept a male pin.



Figure 4.1.33 Male to Female Jumper Wire

Male-to-male jumper wires are the most common type of jumper wires. Each end of the wire has an exposed metal pin that fits into female sockets, such as those found on breadboards or microcontrollers. These wires facilitate quick circuit assembly and allow multiple components to be interconnected without requiring soldering.

Male-to-female jumper wires provide versatility by allowing connections between male and female headers. They are widely used in scenarios where a direct male-to-male connection is not possible.

COMMON USES INCLUDE

- Connecting sensors/modules with female pin headers to microcontrollers.
- Interfacing external devices to breadboards.
- Extending connections when a male pin is required on one end.

FEMALE TO FEMALE JUMPER WIRES

Female-to-female jumper wires are used when two male headers need to be connected. They are useful when dealing with microcontroller boards, sensors, and modules that have male header pins.

COMMON APPLICATIONS INCLUDE

- Connecting two development boards or modules.
- Extending male header pins on circuits.
- Providing flexibility in connecting electronic components without direct soldering.



Figure 4.1.34 Female to Female jumper wire

These wires are a temporary, easy-to-use connection between components, thus eliminating soldering in most cases. The male to female jumper wires are widely used in breadboarding and prototyping as they offer flexibility, reusability, and ease of modification when testing different circuit configurations.

JUMPER WIRES AND ITS WORKS

Jumper wires function by creating temporary electrical connections between different components in a circuit. The conductive core of the wire allows electrical current to pass, enabling communication between devices. The connectors ensure secure placement and transmission of signals without the need for permanent fixtures.

JUMPER WIRES ARE WIDELY USED IN:

- Prototyping electronic circuits on breadboards.
- Debugging and troubleshooting electrical components.
- Learning and experimenting with microcontroller projects.
- Rapidly testing ideas before committing to soldered connections.

By selecting the appropriate type of jumper wire, electronic enthusiasts and engineers can build and test circuits with ease, flexibility, and efficiency.

BREADBOARD

A breadboard is a reusable platform used for building, testing, and prototyping electronic circuits without soldering. It provides a simple and efficient way to connect electronic components temporarily to verify their functionality before making permanent connections.

Breadboards are widely used by hobbyists, students, and engineers for learning and prototyping electronic projects. The board consists of multiple tiny holes arranged in a specific pattern, allowing components to be inserted and connected using jumper wires.

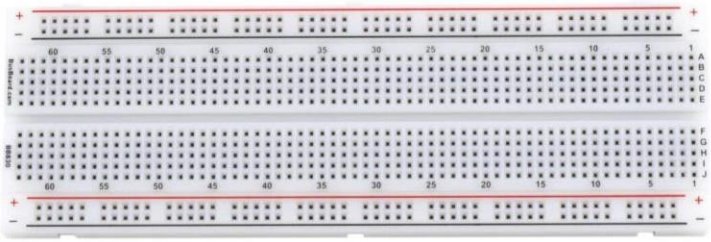


Figure 4.1.3 Bread Board

ARCHITECTURE OF A BREADBOARD

The architecture of a breadboard consists of several interconnected sections, each serving a distinct purpose:

PLASTIC BODY

- The outer casing of the breadboard is made of plastic, usually with a grid of holes.
- Inside, the breadboard has metal strips that provide electrical connectivity.

HORIZONTAL POWER RAILS

- Located on the top and bottom sides of the breadboard.
- Used for providing power (VCC and GND) to the circuit.
- These rails are typically marked with red (+) and blue (-) lines.

VERTICAL TERMINAL STRIPS

- The central section of the breadboard, where most of the circuit connections take place.
- Each column of holes is internally connected, forming groups that allow components to share the same electrical path.

MIDDLE GROOVE (DIP SUPPORT)

- A central dividing groove is present, which isolates two halves of the breadboard.
- It allows integrated circuits (ICs) to be placed with separate connections for each row.

METAL STRIPS

- Beneath the plastic shell, metal strips provide electrical conductivity.
- These strips establish connections between components without the need for soldering.

ADHESIVE BACKING

- Some breadboards have an adhesive layer at the back for easy attachment to surfaces or circuit boards.

TYPES OF BREADBOARDS

Breadboards come in various sizes and configurations, depending on their intended use. The most common types include:

FULL-SIZE BREADBOARD

- Typically consists of 830 holes, providing ample space for complex circuits.
- Includes multiple power rails for efficient power distribution.
- Suitable for large-scale projects and advanced prototyping.

HALF-SIZE BREADBOARD

- Contains around 400 holes, making it compact and portable.
- Used for mid-sized projects and learning purposes.

MINI BREADBOARD

- Small-sized with approximately 170 holes.
- Ideal for small circuits, testing, and minimal component connections.

SELF-ADHESIVE BREADBOARD

- Comes with a sticky back, allowing it to be mounted on surfaces or permanent circuit boards.
- Reduces movement during prototyping.

SOLDERABLE BREADBOARD (PERFBOARD)

- A version that allows for permanent soldered connections.
- Used when transitioning from a temporary prototype to a permanent circuit.

APPLICATIONS OF BREADBOARDS

Breadboards are extensively used in various domains of electronics, including:

PROTOTYPING AND TESTING

- Enables quick and efficient testing of new circuit designs before final implementation.

EDUCATIONAL PURPOSES

- Commonly used in schools and universities for teaching electronics.

MICROCONTROLLER-BASED PROJECTS

- Ideal for working with Arduino, Raspberry Pi, and ESP8266-based projects.

DIY ELECTRONICS

- Helps hobbyists create and modify electronic circuits without permanent wiring.

SENSOR AND ACTUATOR INTEGRATION

- Provides a platform to connect and test various sensors, motors, and other components.

ADVANTAGES OF USING BREADBOARDS

Breadboards offer numerous benefits, making them a preferred choice for prototyping:

NO SOLDERING REQUIRED

- Components can be easily inserted and removed, making modifications simple.

REUSABLE

- Unlike soldered circuits, breadboards can be used multiple times for different projects.

COST-EFFECTIVE

- Affordable and widely available, making them ideal for beginners and professionals.

QUICK AND EASY PROTOTYPING

- Reduces the time needed to assemble and test circuits.

FLEXIBILITY

- Can be used for various circuit designs, from simple LED circuits to complex microcontroller projects.

DISADVANTAGES OF BREADBOARDS

Despite their advantages, breadboards have some limitations:

LIMITED CURRENT HANDLING

- Breadboards cannot handle high-power circuits due to the thin metal strips inside.

LOOSE CONNECTIONS

- Components can become loose over time, leading to unreliable circuits.

SIGNAL INTERFERENCE

- Not ideal for high-frequency circuits, as the connections may introduce unwanted noise and interference.

SIZE CONSTRAINTS

- Large circuits may require multiple breadboards, leading to complex and messy wiring.

BREADBOARDS AND ITS WORK

The working of a breadboard relies on its internal metal strips, which conduct electricity between inserted components. Here's how it functions:

POWER DISTRIBUTION

- Power rails are used to distribute voltage and ground throughout the circuit.
- Components like resistors, LEDs, and sensors are connected to these rails.

COMPONENT PLACEMENT

- Electronic components are inserted into the vertical strips.
- Integrated circuits are placed across the central groove to separate their pins.

ESTABLISHING CONNECTIONS

- Jumper wires connect different sections of the breadboard.
- Signals are transmitted through the metal strips inside the board.

CIRCUIT FUNCTIONALITY

- Once all connections are made, power is applied.
- The components interact according to the circuit design.

Breadboards play a crucial role in electronics prototyping, providing an easy and efficient way to design and test circuits. Understanding their types, working principles, and architecture helps in selecting the right breadboard for various applications. Whether for educational purposes or professional projects, breadboards remain an essential tool in circuit design and development.

SYSTEM ANALYSIS

EXISTING SYSTEM

The existing train safety system provides various conveniences, but safety remains a significant concern. Fire accidents, especially during festivals or hot seasons, can cause severe damage. To mitigate this risk, fire alert sensors have been installed. However, these accidents still occur due to the ineffective placement of sensors. Currently, sensors are positioned near the bathroom and doors, and their functionality remains uncertain.

DRAWBACKS OF THE EXISTING SYSTEM

- Although fire sensors are installed, their placement is limited to areas near the bathrooms and doors. Each compartment has only one sensor, and its operational status is often unknown.
- In the event of a fire outbreak, there is no immediate alert system to inform passengers, leaving them uncertain about how to evacuate safely.
- When a compartment catches fire, passengers typically pull the emergency brake chain, but until then, neither the train crew nor other passengers are aware of the fire's exact location.

PROPOSED SYSTEM

The proposed system aims to enhance fire detection efficiency by improving sensor placement. This system ensures that fire detection occurs regardless of the fire's location within the compartment. Once detected, an alert message is immediately displayed to the loco pilot, and critical information is sent to the Train Traveller Regulators (TTR).

ADVANTAGES OF THE PROPOSED SYSTEM

- The proposed system enhances fire detection by installing additional sensors in each row of a compartment. This ensures that even a small fire outbreak is promptly detected.

- An alert message is instantly displayed for the loco pilot and transmitted to the TTR, facilitating an immediate response.
- Although the system may be costlier, it significantly improves passenger safety during travel.
- As per railway safety protocols, the loco pilot may reduce train speed in emergencies, preventing passengers from panicking and attempting dangerous evacuations.

SYSTEM DESIGN

6.1 BLOCK DIAGRAM

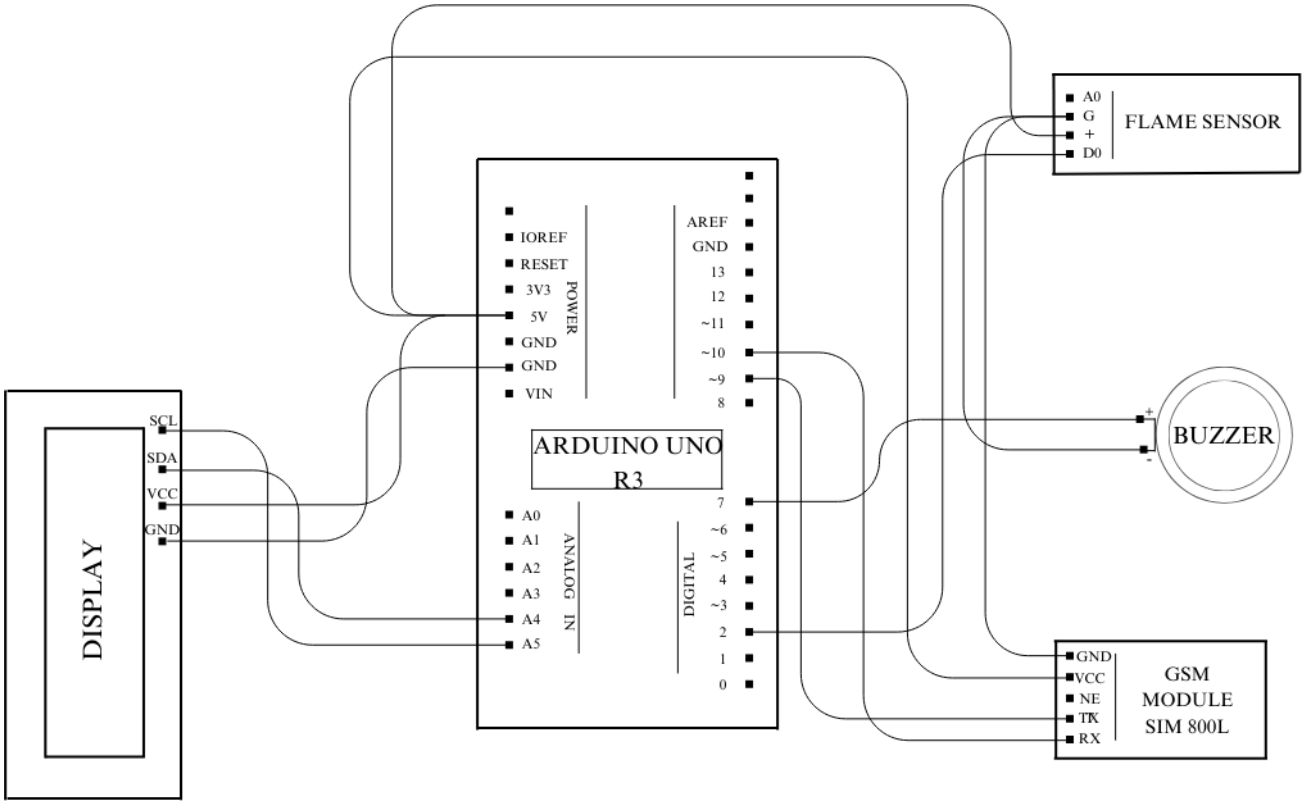


Figure 6.1.1 Circuit Diagram

- A0 To A5: Analog Input
- D1 To D13: Digital pins
- GND: Ground
- VCC: Voltage at the Common Collector
- SDL: Serial data line
- TX: Transmit (sending data)
- RX: Receive (receiving data)
- ARef : Analog REFerence
- 5v: 5 Voltage
- 3v3: 3 Voltage
- SCL: Serial clock line

6.2 WORK FLOW DIAGRAM

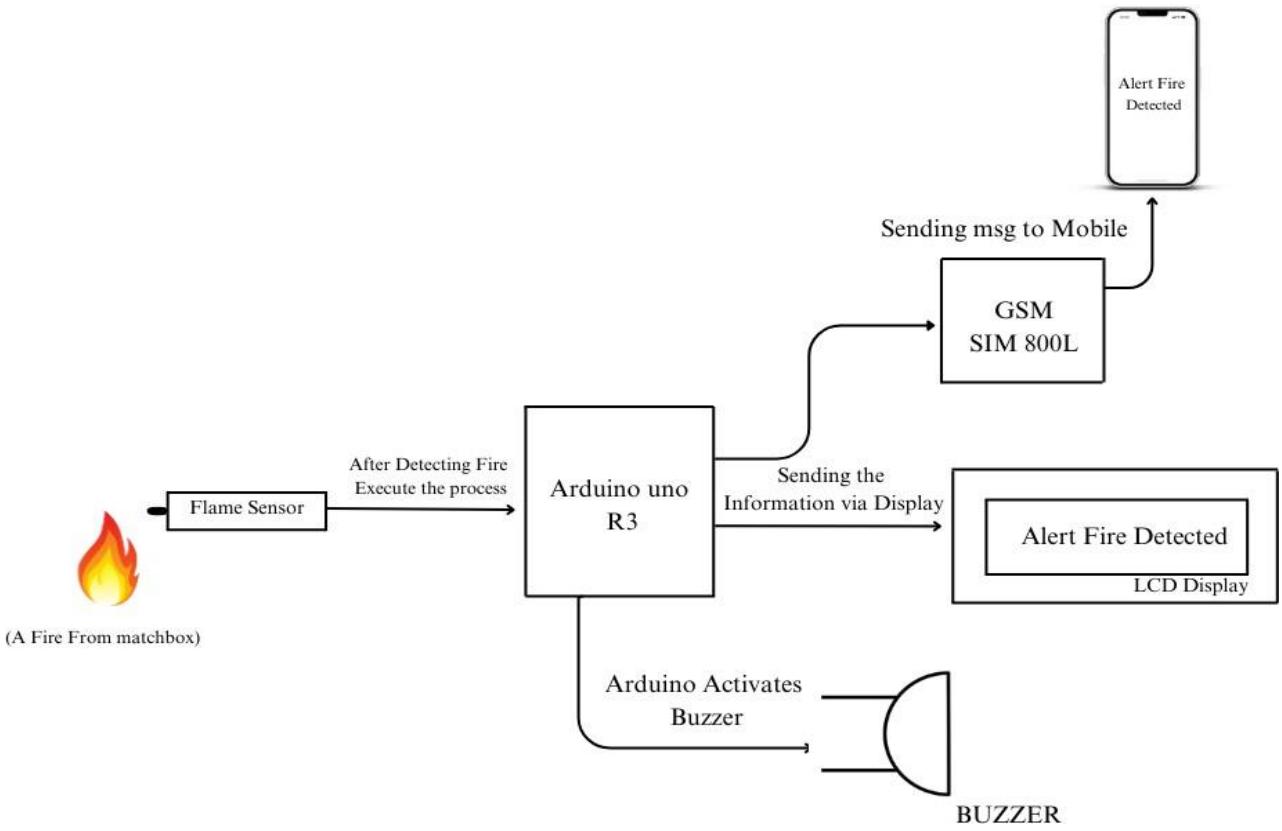


Figure 6.2.1 Signal passing workflow

SYSTEM IMPLEMENTATION

Fire Alert system in train consists of several modules each serving a specific purpose in the overall functionality of the system. The implementation of a robust fire detection and alert system in train compartments is essential for enhancing passenger safety and ensuring swift response to fire emergencies. The proposed system overcomes the limitations of existing fire detection mechanisms by strategically positioning multiple flame sensors, integrating real-time alerts, and enabling immediate action by train operators. This document outlines the systematic implementation of the proposed system, detailing hardware integration, operational workflow, and safety measures.

SENSOR PLACEMENT STRATEGY

Unlike conventional systems where fire sensors are restricted to bathrooms and door areas, the proposed system deploys multiple sensors throughout the compartment to maximize coverage. This ensures that fire incidents are detected regardless of their point of origin, significantly improving response time and safety measures.

PLACEMENT GUIDELINES:

- Sensors are installed at equal intervals across the compartment, ensuring optimal detection capabilities.
- Additional sensors are placed near potential high-risk areas such as electrical panels and luggage racks.
- The placement is optimized to minimize blind spots and maximize early fire detection efficiency.

FIRE DETECTION AND ALERT MECHANISM

The fire detection and alert system follows a well-structured and automated process to ensure timely notifications and prompt action in the event of a fire hazard within a train compartment. The alert mechanism is designed to provide multiple levels of notification, ensuring that all necessary personnel are informed immediately.

Step 1: Fire Detection and Signal Transmission

- The flame sensors operate continuously, scanning the environment for any signs of fire or abnormal heat signatures.
- When a potential fire is detected, the sensor promptly transmits an electrical signal to the Arduino Uno microcontroller, initiating the alert sequence.

Buzzer Activation – Audible Emergency Alarm

- A high-decibel buzzer is triggered to produce a loud and continuous alarm within the affected compartment.
- This immediate audio alert warns passengers and onboard staff of the fire hazard, prompting swift action.

LCD Notification – Visual Emergency Display

- The 16x2 LCD screen prominently displays a real-time emergency message: "FIRE DETECTED! ALERT!"
- The message remains on the screen until the fire is confirmed to be under control, ensuring clear and visible communication to onboard personnel, including the loco pilot and TTR (Train Traveler Regulators).

GSM Alert Message – Remote Notification System

The SIM800L GSM module is programmed to send an automatic SMS notification to designated recipients, including the TTR, ensuring that key personnel are informed without delay.

The SMS includes specific details such as:

- Compartment number where the fire is detected
- Fire status indicating whether the fire is active or escalating
- Timestamp of the alert for record-keeping and analysis

This ensures that railway authorities can take appropriate safety measures immediately, including coordinating emergency response teams at the next station if necessary.

Step 2: Response and Emergency Coordination

Loco Pilot Action: Upon receiving the alert, the loco pilot promptly initiates safety protocols, which may include slowing down the train, stopping at the nearest station, or activating onboard fire suppression systems if available.

TTR Coordination: The Train Traveler Regulators (TTR) receives the alert in real-time, allowing them to coordinate emergency response teams, communicate with railway officials, and prepare firefighting resources at upcoming stations.

SYSTEM TESTING

A comprehensive testing process ensures the fire detection and alert system operates efficiently under various conditions. The following structured test cases will be conducted to validate the system's functionality, reliability, and efficiency in real-world scenarios. Proper testing helps in identifying potential issues before deployment, ensuring that the system meets all necessary safety and performance standards.

IOT PROJECT TESTING

IoT (Internet of Things) project testing in documentation refers to the process of verifying and validating the functionality, performance, and reliability of an IoT system before deployment. This ensures that the system works as expected under different conditions. In documentation, this section provides detailed test cases, results, and observations for future reference.

KEY ASPECTS OF IOT PROJECT TESTING DOCUMENTATION

1. FUNCTIONAL TESTING

Ensures all components (sensors, actuators and communication modules) work correctly.

Example: Checking if the **flame sensor** in your fire detection system detects fire and triggers alerts properly.

2. CONNECTIVITY TESTING

Verifies that IoT devices can communicate over networks (Wi-Fi, GSM, Bluetooth, etc.).

Example: Checking if the **SIM800L module** sends SMS alerts when fire is detected.

3. PERFORMANCE TESTING

Measures system response time, data processing speed, and reliability under load.

Example: Testing how fast the **Arduino Uno** processes sensor data and sends an alert.

4. SECURITY TESTING

Ensures data is transmitted securely and the system is protected against unauthorized access.

Example: Checking if the GSM module can only send messages to pre-configured numbers.

5. POWER CONSUMPTION TESTING

Evaluates the energy efficiency of the system.

Example: Measuring the power usage of the Arduino Uno, SIM800L, and buzzer under different conditions.

SENSOR DETECTION TEST

The objective of this test is to verify that the flame sensor accurately detects fire and triggers the appropriate response. A controlled flame will be introduced near the sensor to observe its output using a serial monitor. The Arduino will be checked to ensure it correctly interprets and processes the fire detection. The test will be repeated multiple times under different conditions, such as varying light intensities and angles. The accuracy of the sensor will be recorded by comparing actual detections with expected results. The system must ensure that false positives are minimized and that the sensor does not respond to non-fire sources.

BUZZER ACTIVATION TEST

This test ensures that the buzzer activates immediately upon fire detection, providing an effective alert for passengers. The flame sensor will be manually triggered, and the buzzer's response time will be measured to confirm instant activation. The buzzer's sound levels will be tested in different noise environments to ensure it is loud enough to be heard clearly.

Additionally, the durability of the buzzer will be evaluated by testing its performance under prolonged use and repeated activations.

LCD DISPLAY FUNCTIONALITY TEST

To verify that the LCD displays the correct alert message during a fire event, the flame sensor will be triggered, and the displayed message will be examined. The test will check for proper message display, such as “Fire Detected! Take Action,” and assess any potential lag, incorrect characters, or malfunctions. The LCD’s visibility will be tested under different lighting conditions, and it will be ensured that the alert remains on screen until the issue is resolved.

GSM MODULE COMMUNICATION TEST

This test ensures that the GSM module successfully sends SMS alerts to the designated recipients. The flame sensor will be triggered, and the transmission process will be observed to verify that the loco pilot and Train Traveler Regulators (TTR) receive the alert. The system will be tested under different network conditions, including low signal, high signal, and no signal, to assess message delivery reliability. The time taken for message transmission will be measured, and the system’s ability to resend messages if initial delivery fails will be tested.

POWER SUPPLY STABILITY TEST

To validate the system’s reliability under different power conditions, the system will be operated under varying voltage levels, and simulated power fluctuations will be introduced. The test will check whether the system remains operational without resetting unexpectedly. If applicable, backup power options will be evaluated. Energy consumption will also be measured under prolonged use to ensure system efficiency.

9.1 CODING

PACKAGES AND LIBRARIES USED

Purpose: This library is used for I2C communication, allowing the microcontroller to communicate with devices such as the LCD.

Functionality: It enables data transmission over the I2C protocol and simplifies multi-device communication on the same bus.

The following code is designed for an Arduino-based fire detection and alert system using an IR sensor, a buzzer, an LCD display, and a GSM module. The system continuously monitors for fire presence and sends an alert message via SMS when a fire is detected.

LIBRARY INCLUSIONS

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <SoftwareSerial.h>
```

- #include <Wire.h> – Enables communication with I2C devices, such as the LCD module.
- #include <LiquidCrystal_I2C.h> – Provides functions to interface with an I2C-based LCD display.
- #include <SoftwareSerial.h> – Allows software-based serial communication with the GSM module using digital pins.

PIN DEFINITIONS

```
#define IR_SENSOR 2                    // IR sensor connected to pin 2

#define BUZZER 7                       // Buzzer connected to pin 7
```

- #define IR_SENSOR 2 – Assigns pin 2 to the IR sensor.

- #define BUZZER 7 – Assigns pin 7 to the buzzer.

OBJECT INITIALIZATION

```
LiquidCrystal_I2C lcd(0x27, 16, 2);

SoftwareSerial gsm(9, 10);                    // TX=9, RX=10 (for GSM module)
```

- LiquidCrystal_I2C lcd(0x27, 16, 2); – Initializes an LCD display with I2C address 0x27, 16 columns, and 2 rows.
- SoftwareSerial gsm(9, 10); – Defines software serial communication for the GSM module using TX on pin 9 and RX on pin 10.

SETUP FUNCTION (RUNS ONCE AT STARTUP)

```
void s+etup() {

  pinMode(IR_SENSOR, INPUT);

  pinMode(BUZZER, OUTPUT);
```

- pinMode(IR_SENSOR, INPUT); – Configures the IR sensor pin as an input.
- pinMode(BUZZER, OUTPUT); – Configures the buzzer pin as an output.

```
Serial.begin(9600);

Wire.begin();

lcd.init();
lcd.backlight();
```

- Serial.begin(9600); – Starts serial communication at a baud rate of 9600.
- Wire.begin(); – Initializes I2C communication for the LCD.
- lcd.init(); – Initializes the LCD display.
- lcd.backlight(); – Turns on the LCD backlight.

```
gsm.begin(9600);
```

- gsm.begin(9600); – Initializes GSM communication at 9600 baud rate.

```
Serial.println("Initializing GSM...");
 lcd.setCursor(0, 0);
 lcd.print("GSM Initializing");
delay(3000);
```

- Displays “GSM Initializing” on the LCD and waits for 3 seconds.

```
gsm.println("AT");
delay(1000);
gsm.println("AT+CMGF=1");                    // Set SMS mode to text
delay(1000);
gsm.println("AT+CSCS=\"GSM\"");
delay(1000);
```

- Sends AT commands to configure the GSM module for SMS communication.

```
lcd.clear();
   Serial.println("System  Ready");
}
```

- Clears the LCD and prints “System Ready” to indicate that initialization is complete.

MAIN LOOP (RUNS REPEATEDLY)

```
void loop() {
   int sensorValue = digitalRead(IR_SENSOR);
```

- Reads the IR sensor value and stores it in sensorValue.

```
if (sensorValue == LOW) { // Object detected
      Serial.println("Object Detected!");
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("Fire is Detected");
```

- If the IR sensor detects an object (fire), the LCD displays “Fire is Detected.”

```
digitalWrite(BUZZER, HIGH);                    // Turn ON buzzer
delay(500);
digitalWrite(BUZZER, LOW);                     // Turn OFF buzzer after beep
```

- The buzzer sounds for 500ms and then turns off.

```
sendSMS("Alert: Fire Is Detected!");
      delay(5000);
```

- Calls the sendSMS function to notify authorities via SMS.

```
} else {
      lcd.clear();
```

```
    lcd.setCursor(0, 0);
    lcd.print("No Object");
    Serial.println("Fire is not detected.");
    digitalWrite(BUZZER, LOW);
  }
  delay(500);
}
```

- If no fire is detected, the LCD displays “No Object.”

SMS SENDING FUNCTION

```
void sendSMS(String message) {
Serial.println("Sending SMS...");
```

- Prints “Sending SMS...” to the serial monitor.

```
gsm.println("AT+CMGS=\"+918248112514\"");          // Replace with your phone number
  delay(1000);
  gsm.print(message);
  delay(1000);
  gsm.write(26); // CTRL+Z to send message
  delay(5000);
```

- Sends an SMS with the message “Alert: Fire Is Detected!” to the specified number.

Serial.println("SMS Sent.");}

- Confirms that the SMS has been sent successfully.

9.2 SCREENSHOT

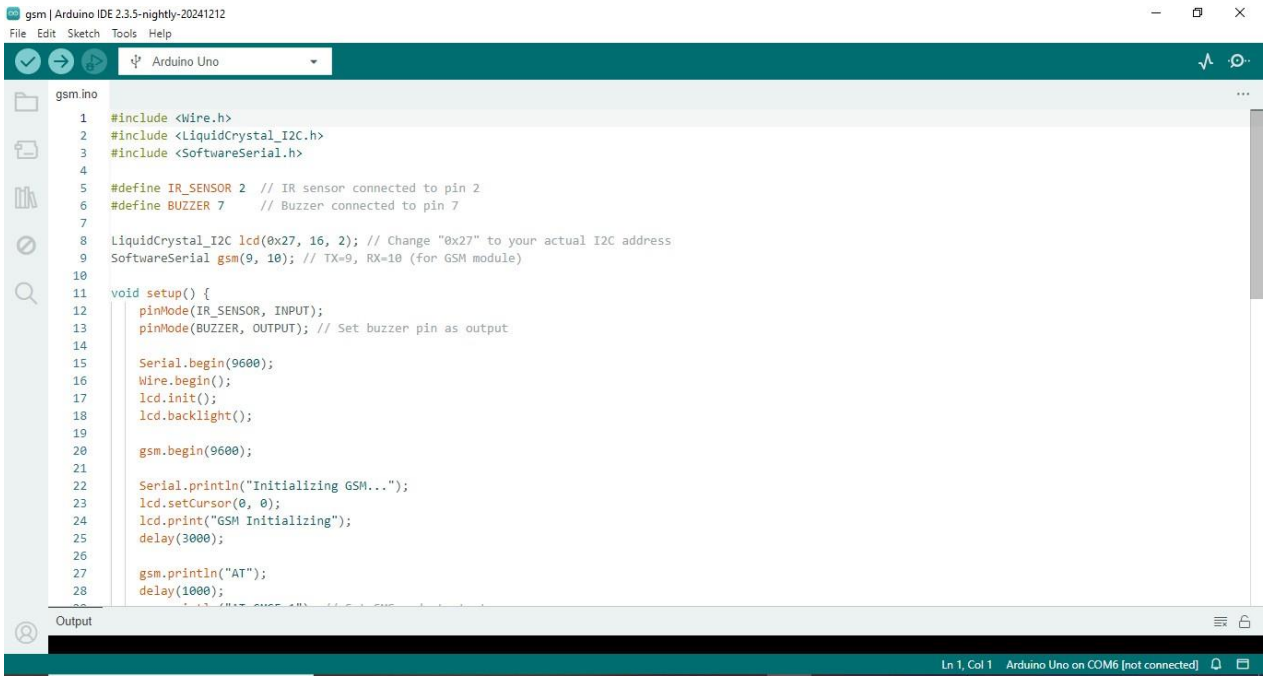


Figure 9.2.1 Initializing Coding

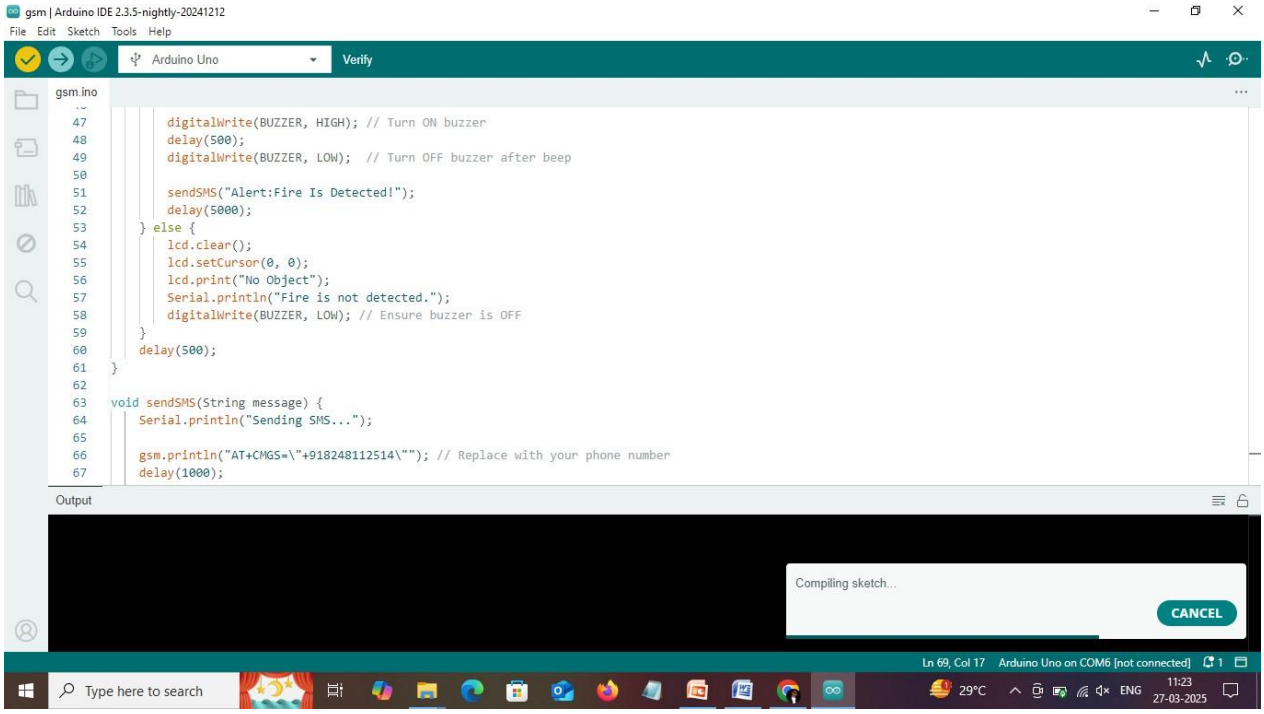


Figure 9.2.2 Compiling Coding

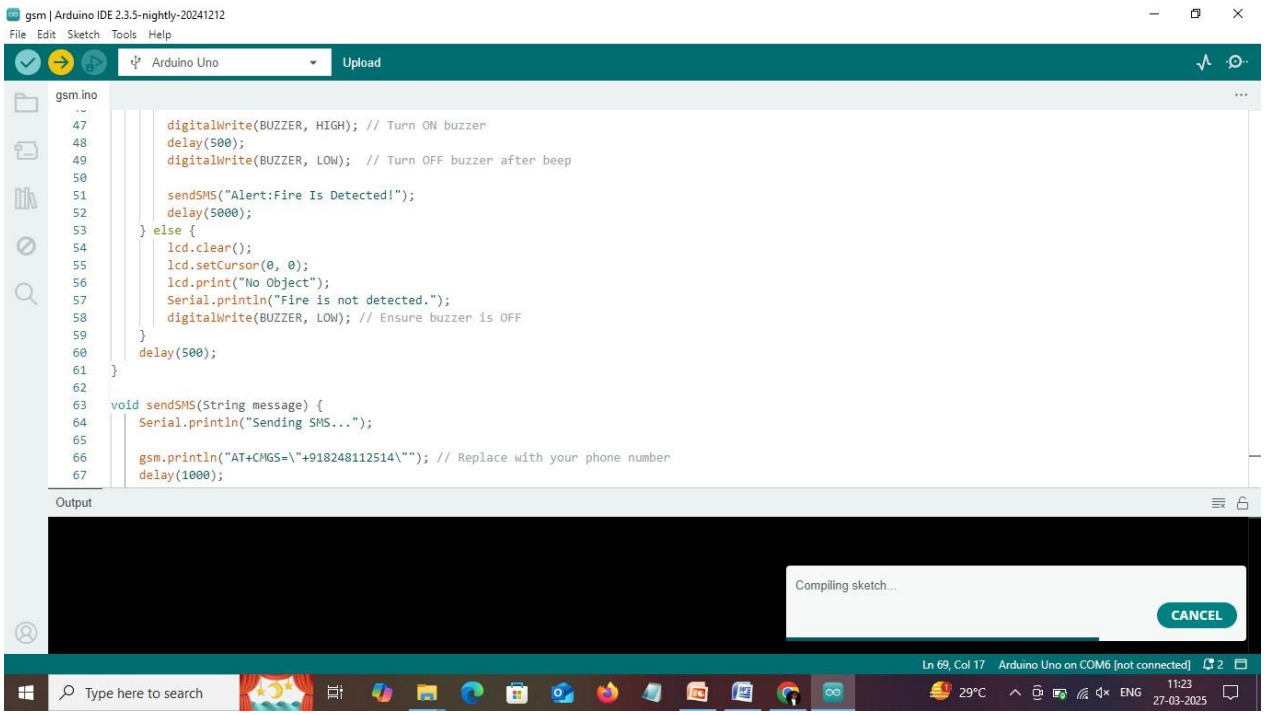


Figure 9.2.3 Uploading the Code to Arduino Board

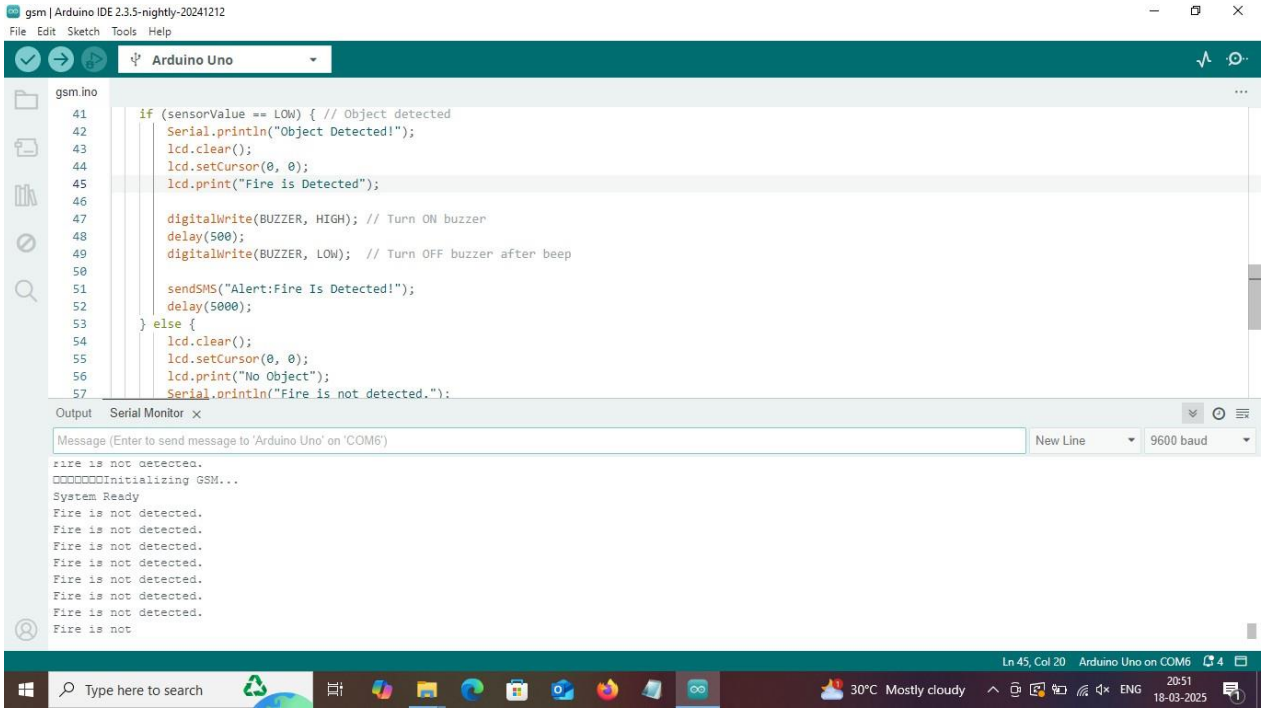


Figure 9.2.4 The system was Ready to detect the fire



Figure 9.2.5 Initializing the GSM



Figure 9.2.6 The Display For Loco Pilot



Figure 9.2.7 A Flame Sensor detecting The Fire

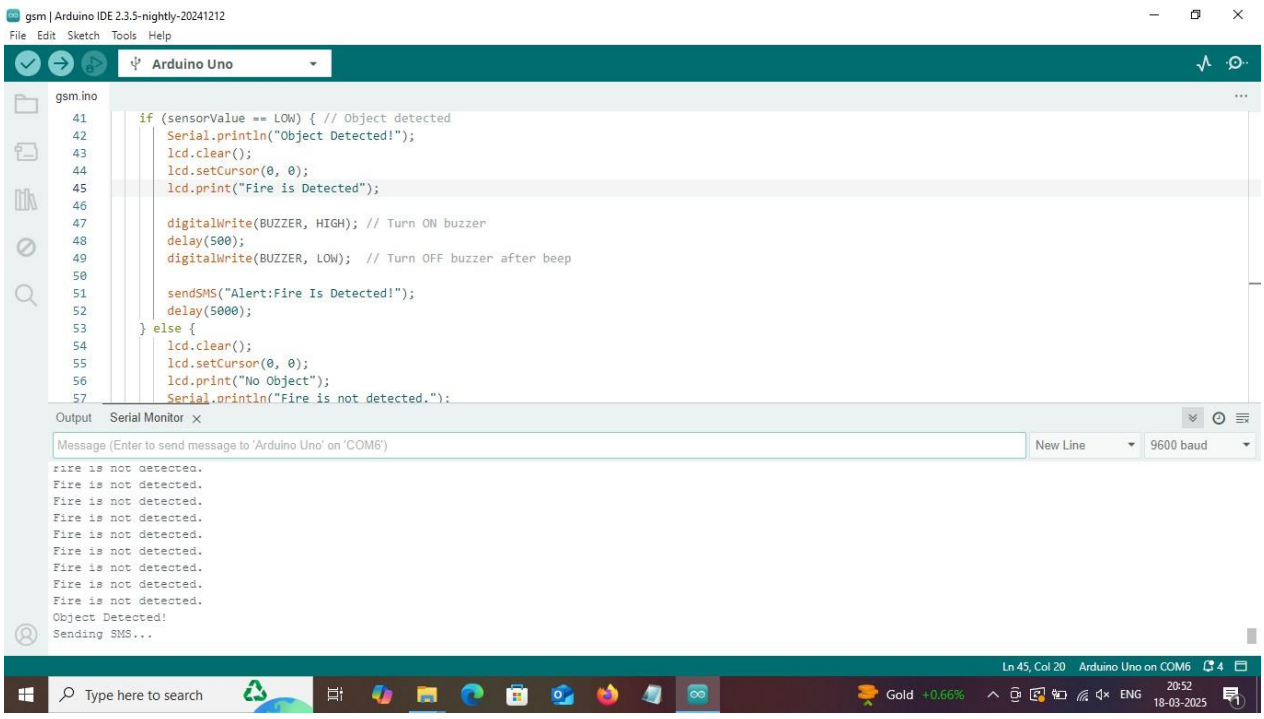


Figure 9.2.8 After Detecting the Fire System Preparing to send the information to TTR & Loco Pilot

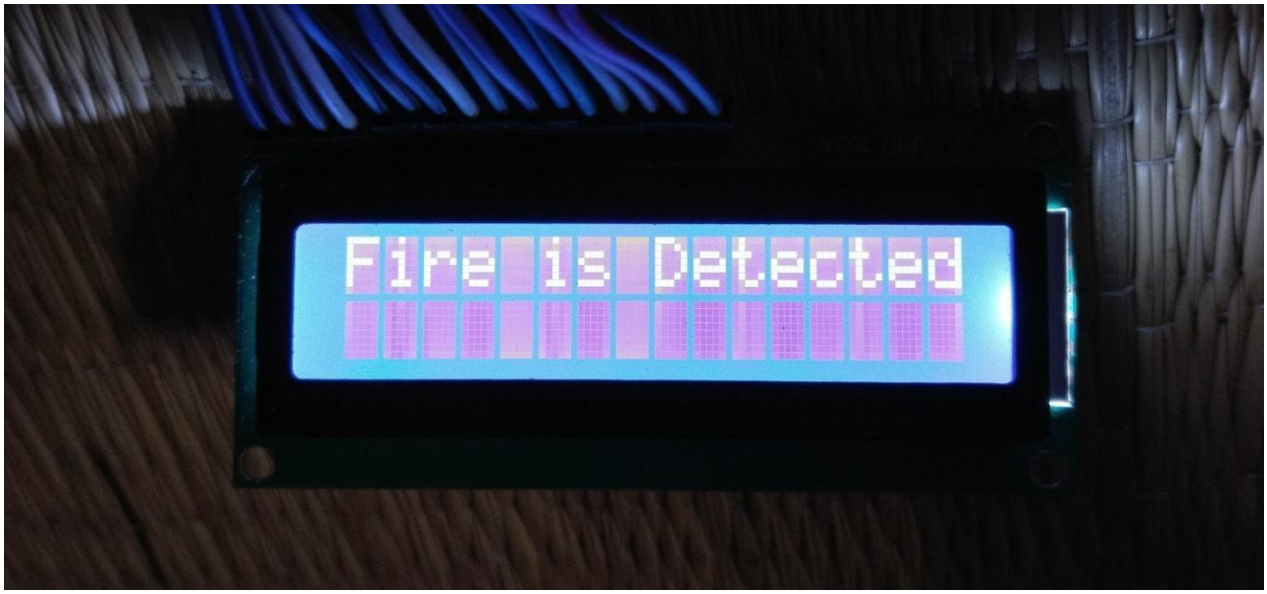


Figure 9.2.9 Alert Display For Loco Pilot

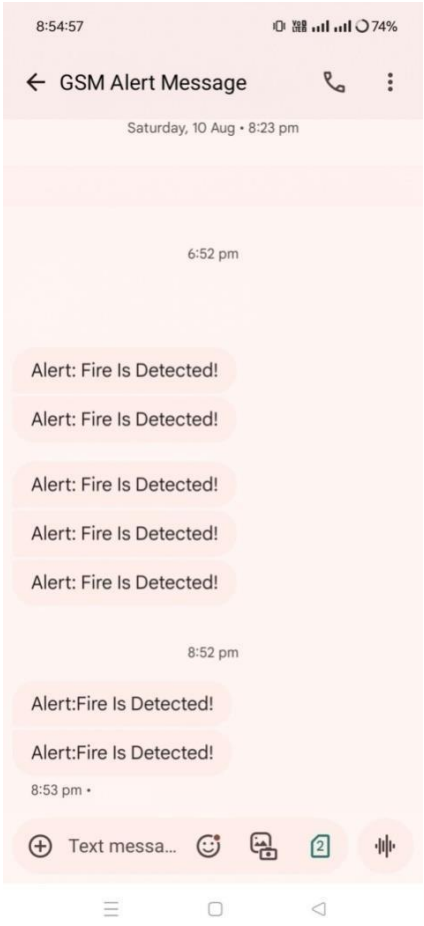


Figure 9.2.10 Alert Message For TTR (Train Traveller Regulators)

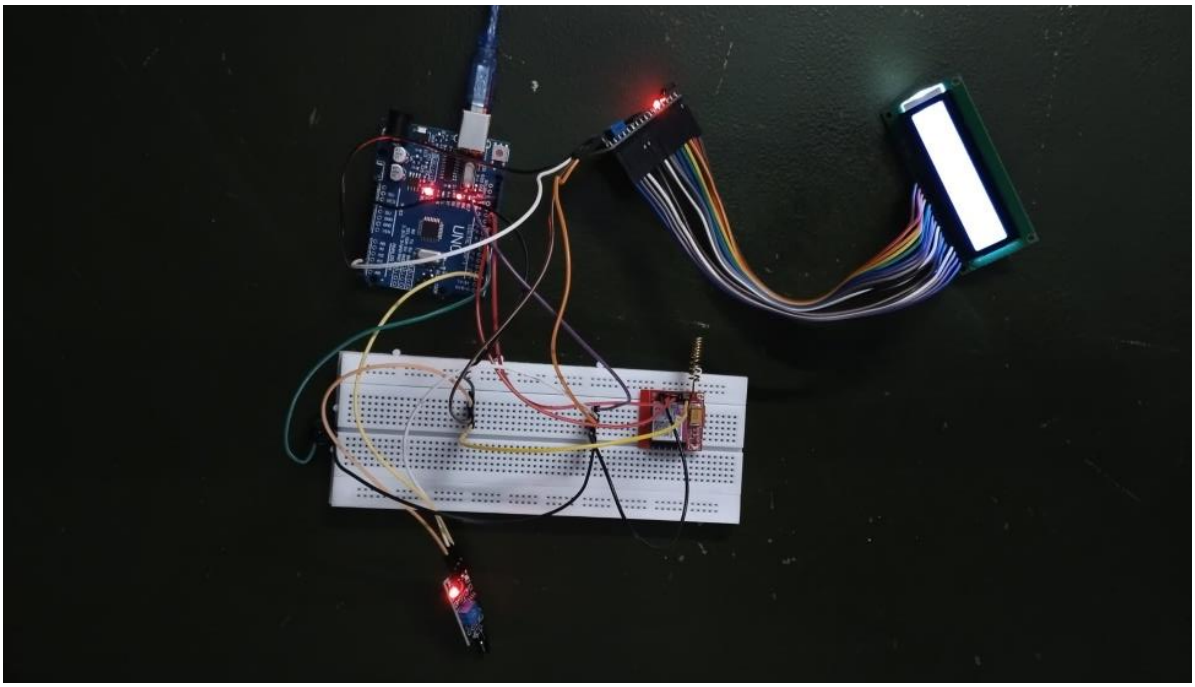


Figure 9.2.11 Final connection

CONCLUSION

The implementation of a compartment-wise, intelligent fire detection and alert system significantly enhances railway safety by ensuring rapid and accurate fire detection, followed by an immediate response mechanism. This innovative project serves as a groundbreaking advancement in railway fire safety, providing a proactive solution to minimize fire-related risks in train travel. By enabling real-time alerts and swift action, this system plays a crucial role in safeguarding passengers, reducing potential damage, and preventing catastrophic incidents, ultimately protecting countless lives and valuable property.

FUTURE ENHANCEMENT

To further enhance fire safety in railways, the system can incorporate advanced technologies for improved detection and response. Future upgrades may include the integration of smoke sensors alongside flame sensors to enhance detection capabilities. Additionally, GPS tracking can provide precise location data, enabling faster emergency response and ensuring passenger safety. By leveraging technological advancements, Indian Railways can significantly reduce fire hazards, creating a safer and more secure travel experience. Implementing this system is not just an improvement but a necessity for modern railway operations.

BIBLIOGRAPHY

BOOK REFERENCES

- J. Blum, Exploring Arduino: "Tools and Techniques for Engineering Wizardry", 2nd ed. New York, NY, USA: Wiley, 2019.
- S. Monk, Programming Arduino: "Getting Started with Sketches", 2nd ed. New York, NY, USA: McGraw-Hill Education, 2020.
- Gullybaba.com Panel. (2024). MCS-231 Mobile Computing (Revised ed.).
- Sauter, M. (2021). From GSM to LTE-Advanced Pro and 5G:" An Introduction to Mobile Networks and Mobile Broadband "(4th ed.).
- P. Raj and A. C. Raman, "The Internet of Things: Enabling Technologies, Platforms, and Use Cases", 1st ed. Boca Raton, FL, USA: CRC Press, 2024.

WEBSITE REFERENCES

- https://www.arduino.cc/en/about
- https://www.geeksforgeeks.org/gsm-in-wireless-communication
- https://www.techtarget.com
- https://docs.sunfounder.com/projects/ultimate-sensor-kit/en/latest/components_basic/03-component_flame.html
- https://docs.arduino.cc/libraries/liquidcrystal
- https://docs.sunfounder.com/projects/sfcomponents/en/latest/component_wires.html
- https://electronicssecret.wordpress.com/2022/07/14/jumper-wires-an-introduction-applications-and-working-principles
- https://docs.sunfounder.com/projects